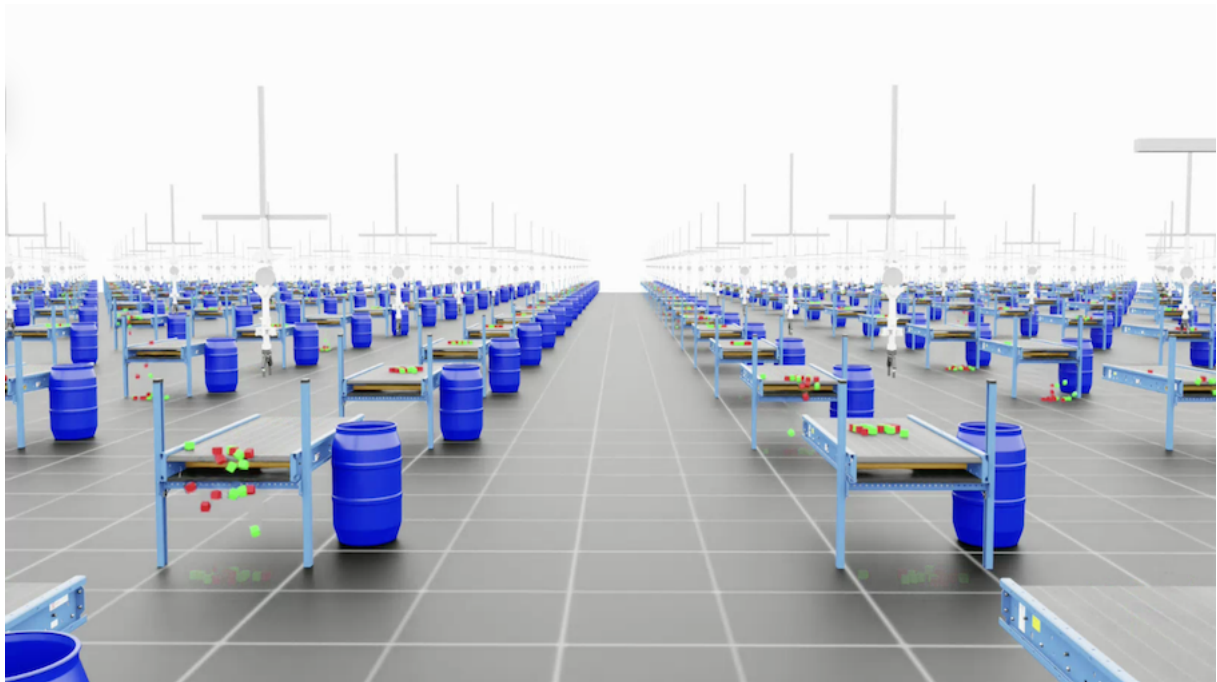


Simulative Construction and Verification of a Tendon-Driven Robot for Conveyor-Based Pick-and-Place Tasks Using Isaac Lab

Project Thesis im Studiengang Computational Engineering M.
Sc.

Friedrich-Alexander-Universität Erlangen-Nürnberg
Lehrstuhl für Fertigungsautomatisierung und Produktionssystematik
Prof. Dr.-Ing. J. Franke



Bearbeiter: Georg Meyer

Matrikelnr.: 22791103

Betreuer: Prof. Dr.-Ing. J. Franke
Dipl.-Ing. M. Landgraf

Abgabetermin: 01.07.2026

Bearbeitungszeit: x Monate

Erklärung

Ich versichere, dass ich die vorliegende Arbeit ohne fremde Hilfe und ohne Benutzung anderer als der angegebenen Quellen angefertigt habe und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen hat und von dieser als Teil einer Prüfungsleistung angenommen wurde. Alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, sind als solche gekennzeichnet.

Erlangen, den 9. März 2026

Georg Meyer

Contents

List of Figures

List of Tables

List of Abbreviations

API	Application Programming Interface
CAD	Computer-Aided Design
CPU	Central Processing Unit
DoF	Degree of Freedom
DR	Domain Randomization
dVRK	da Vinci Research Kit
FEM	Finite Element Method
FK	Forward Kinematics
GAE	Generalised Advantage Estimation
GPU	Graphics Processing Unit
IK	Inverse Kinematics
IL	Imitation Learning
MDP	Markov Decision Process
MJCF	MuJoCo XML Format
MPC	Model Predictive Control
MPM	Material Point Method
PBD	Position-Based Dynamics
PD	Proportional-Derivative
PGS	Projected Gauss–Seidel
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimization
RGB-D	Red-Green-Blue plus Depth
RL	Reinforcement Learning
ROS	Robot Operating System
SDF	Signed Distance Field
TGS	Temporal Gauss–Seidel
URDF	Unified Robot Description Format
USD	Universal Scene Description
VBD	Vertex Block Descent
XML	Extensible Markup Language
XPBD	Extended Position-Based Dynamics

List of Symbols

$\mathcal{A}$	Action space
A_t	Advantage estimator
ϵ	Clipping parameter (PPO)
γ	Discount factor
h	Simulation time step
$J(\theta)$	Expected discounted return
k_j	Stiffness parameter (PBD)
$\mathbf{M}$	Mass matrix
π_θ	Policy (parameterized by θ)
$r(s_t, a_t)$	Reward function
$\mathcal{S}$	State space
θ	Policy parameters

1 Introduction

1.1 Motivation and Context

1.2 Problem Statement and Objectives

Autonomous sorting of textiles from mixed waste streams is a challenging manipulation problem: garments such as T-shirts are deformable, exhibit highly variable shapes, and interact with their environment through complex contact dynamics. In industrial recycling contexts, reliable textile extraction is a key prerequisite for downstream steps such as material recovery or further sorting by condition (e.g., damaged vs. reusable). Classical, model-based manipulation pipelines typically rely on accurate state estimation and carefully tuned controllers, which becomes difficult when both the manipulated object and the contact conditions change continuously.

This thesis addresses the problem of learning a robust control policy for a custom tendon-driven robot to selectively retrieve a T-shirt from a conveyor transporting heterogeneous waste items and to deposit it into a designated bin. The employed robotic system is realized in NVIDIA Isaac Sim and consists of three main subsystems: (i) a central tendon-driven arm section derived from an existing tendon-robot design, (ii) an articulated base that provides additional linear actuation in the y - and z -directions to expand the reachable workspace and to improve access to items on the conveyor, and (iii) a Robotiq gripper as the end-effector. The control policy is trained in IsaacLab using Proximal Policy Optimization implemented with `skrl`. The overall learning objective is defined as a selective pick-and-throw behavior: *grasp the T-shirt, lift it from the moving belt, and reliably eject it into the target bin while leaving non-target waste items behind.*

The scope of this work focuses on the learning-based manipulation stack in simulation and its preparation for sim-to-real transfer. Perception is treated as an enabling component (e.g., via ground-truth labels in simulation or externally provided detections) rather than the primary research contribution. In the broader project context, this selective retrieval capability forms the foundation for future extensions towards *condition-based* T-shirt sorting, where policies must not only identify garments but also manipulate and present them in ways that enable inspection and quality assessment.

The thesis pursues the following objectives:

1. **High-fidelity simulation environment:** Develop a reproducible Isaac Sim scene that models a conveyor-based waste stream, a target bin, and representative waste items including a deformable T-shirt, with randomized initial conditions and physical parameters to support robust learning.
2. **Robot model integration and validation:** Integrate the custom robot (tendon-driven arm segment, linear y/z base actuation, and Robotiq gripper) into IsaacLab, and verify the correctness of kinematics, actuation interfaces, joint limits, and controller settings required for stable simulation-based training.
3. **MDP formulation of the manipulation task:** Formulate the problem as an Markov Decision Process within the IsaacLab workflow by specifying observation space, action space, reward design, and termination criteria. The formulation must explicitly encode *selectivity* (T-shirt vs. non-target items) and *successful disposal* into the bin.
4. **Curriculum from basic skills to the final task:** Implement a staged training procedure that progresses from foundational behaviors (e.g., reaching, grasping, simple pick-and-place) to the full conveyor-based selective pick-and-throw task, enabling sample-efficient learning and reducing reward sparsity.
5. **Policy training and quantitative evaluation:** Train Proximal Policy Optimization policies using `skrl` and evaluate performance with metrics such as task success rate, false-pick rate (non-target retrieval), missed-pick rate (target left on conveyor), bin placement success, cycle time, and robustness under domain randomization (e.g., belt speed, friction, mass, cloth stiffness).
6. **Sim-to-real readiness and integration pathway:** Assess transfer robustness via noise and parameter randomization and define an executable integration pathway towards real-system deployment (e.g., Robot Operating System 2 interfaces, logging, and safety constraints), identifying remaining gaps that must be closed for real-world operation.
7. **Foundation for condition-based sorting:** Derive requirements for extending the learned manipulation policy towards condition-based T-shirt sorting, including which additional observations, interaction primitives (e.g., stretching/presentation), and evaluation criteria would be needed in subsequent work.

1.3 Thesis Outline

This thesis is structured as follows. Chapter ?? introduces the fundamentals required for this work, covering reinforcement learning for robotic manipulation with an emphasis on Proximal Policy Optimization, deformable-object (cloth) manipulation challenges, and the simulation tools used in this project (Isaac Sim and IsaacLab). Chapter ?? describes the simulated system and task setup in detail: the conveyor scenario with mixed waste items, the selective retrieval objective, and the custom robot model consisting of the tendon-driven arm segment, the additional linear y/z base actuation, and the Robotiq gripper.

Chapter ?? formulates the learning problem as an Markov Decision Process and explains the design of observations, actions, reward terms, and termination conditions to realize selective pick-and-throw behavior (retrieve only the T-shirt and deposit it into a bin). Chapter ?? presents the training procedure in IsaacLab using `skr1` Proximal Policy Optimization, including curriculum design from simpler sub-skills to the full conveyor task and the use of randomization to improve robustness.

Chapter ?? defines the experimental protocol and evaluation metrics, focusing on sorting-relevant criteria such as success rate (T-shirt in bin), false retrievals of non-target objects, misses of the target, and robustness under variations of object and conveyor properties. Chapter ?? reports the quantitative and qualitative results of the learned policies and analyzes typical success behaviors and failure cases.

Finally, Chapter ?? discusses the implications and limitations of the approach, including simulation fidelity and sim-to-real considerations, and Chapter ?? concludes the thesis and outlines future work towards condition-based T-shirt sorting, where retrieval is extended to classification and handling based on garment condition.

2 Background

2.1 Reinforcement Learning for Robotic Manipulation

Robotic manipulation requires generating control commands that achieve a goal while dealing with uncertainty, partial observability, contacts, and high-dimensional configuration spaces. Classical approaches rely on analytical models, trajectory optimisation, and carefully engineered feedback control. However, accurate models of frictional contact, deformable objects, or cluttered interactions are often difficult to obtain, and manually designing robust policies can be time-consuming. Reinforcement Learning provides an alternative: it enables learning control policies from interaction data by optimising a task objective, thereby reducing reliance on explicit modelling and hand-crafted strategies. General Reinforcement Learning background and formalism are treated in standard texts such as Sutton and Barto [?], and robotics-specific perspectives (including practical constraints and common failure modes) are discussed in Kober *et al.* [?].

MDP viewpoint and partial observability. A manipulation task can be modelled as an Markov Decision Process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $P(s_{t+1} \mid s_t, a_t)$ the transition dynamics, $r(s_t, a_t)$ the reward, and $\gamma \in [0, 1)$ the discount factor [?]. A policy $\pi_\theta(a \mid s)$ (parameterized by θ) induces trajectories $(s_0, a_0, r_0, \dots, s_T)$ and is trained to maximize the expected discounted return $J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right]$ [?]. In real manipulation systems, the full state s_t is typically not directly accessible; instead, control is based on observations o_t (e.g., proprioception, gripper state, pose estimates, perception features). This partial observability motivates careful observation design and, where appropriate, augmenting policies with memory (e.g., recurrent networks), or stacking observation histories to approximate the Markov property in practice [?].

Key challenges in manipulation. Three aspects commonly make Reinforcement Learning for manipulation demanding: (i) **continuous, high-dimensional control**, where actions may correspond to joint torques, velocities, or Cartesian increments for many degrees of freedom; (ii) **contact-rich and nonlinear dynamics**, where small

modelling errors in friction or contact impulses can lead to qualitatively different outcomes; and (iii) **sparse or delayed feedback**, where success may only be measurable after long sequences of intermediate steps. Further practical constraints include safety, limited real-world interaction budgets, and the need for robustness across object variations and perception noise [?]. These characteristics influence algorithm choice, reward design, termination conditions, and the role of simulation during training.

Reward design and task specification. In Reinforcement Learning, the reward function encodes the task objective. For manipulation, rewards are often composed of shaping terms (e.g., distance-to-target, alignment, grasp quality proxies) and sparse success signals (e.g., “object lifted above height”). Well-shaped rewards can accelerate learning but risk biasing the learned behavior toward unintended shortcuts; conversely, sparse rewards better reflect the true objective but can make exploration difficult. Robotics Reinforcement Learning workflows therefore often iterate on reward definitions and termination criteria to balance learnability and task fidelity [?, ?].

2.1.0.1 Learning objectives and algorithm families

For continuous control, modern deep Reinforcement Learning methods commonly use *policy gradient* and *actor-critic* approaches. Policy gradient methods directly optimise the policy parameters by ascending an estimate of $\nabla_{\theta} J(\theta)$, which naturally supports continuous actions through differentiable policy parameterizations [?]. Actor-critic methods introduce a learned critic (value function) that provides lower-variance learning signals for the actor, often improving stability and learning speed in practice [?]. A central quantity is the advantage function $A^{\pi}(s_t, a_t)$, which measures how much better an action is compared to the policy’s baseline behaviour at a state. In practice, advantage estimates $\hat{A}_t$ are computed from value predictions and collected rollouts, and then used to weight policy updates [?].

Bias-variance trade-off in advantage estimation. A practical challenge is balancing low-variance learning signals with accurate credit assignment over time. Generalised Advantage Estimation provides a widely used mechanism to interpolate between Monte Carlo returns and bootstrapped temporal-difference estimates by exponentially weighting temporal-difference residuals [?]. This improves learning stability in many continuous-control applications and is commonly paired with actor-critic training.

2.1.0.2 Exploration and data collection in manipulation

A defining challenge in Reinforcement Learning is the *exploration–exploitation trade-off*: the agent must explore sufficiently to discover high-reward behaviours while exploiting learned knowledge to improve performance [?]. In manipulation, exploration is complicated by (i) long horizons, (ii) sparse success signals, and (iii) irreversible or unsafe states (in real systems) [?]. Common practical measures include:

- **Stochastic policies and action sampling**, which induce exploratory behavior while still allowing gradient-based optimization [?].
- **Reward shaping and curricula**, where training starts from simplified conditions (e.g., easier initial object poses) and progressively increases difficulty to maintain a learnable signal [?].
- **Episode resets and termination design**, which affect state visitation and can substantially change the data distribution seen by the learner [?].

These aspects tie the learning algorithm closely to environment design: even a robust optimiser can fail if exploration is insufficient or if the reward/termination specification induces pathological behavior [?, ?].

2.1.0.3 Simulation-to-reality considerations

Reinforcement Learning policies are frequently trained in simulation to avoid wear, risk, and slow data collection on real robots [?]. However, transferring policies to the real world is challenging due to the *reality gap*: discrepancies in geometry, contact parameters, actuator dynamics, sensor noise, and unmodeled effects can degrade performance. A common strategy is **domain randomization**, where visual and physical parameters are randomized during training so that the policy learns behaviors that are robust across a range of plausible conditions [?]. A closely related approach is **dynamics randomization**, which targets uncertainties in physical parameters (e.g., mass, friction, motor gains, delays) to improve transfer for control policies [?]. In practice, these methods motivate designing training environments that reflect not only the nominal task but also a distribution of plausible perturbations, thereby improving robustness of learned manipulation policies when deployed outside the simulator [?, ?, ?].

2.1.0.4 Proximal Policy Optimization (PPO)

Among actor–critic methods, Proximal Policy Optimization is a popular choice due to its robustness and simplicity. Proximal Policy Optimization performs iterative policy im-

provement by collecting trajectories with the current policy and updating the policy with a surrogate objective that discourages overly large steps in policy space [?]. Define the importance ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$. Proximal Policy Optimization maximizes the clipped objective

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)], \quad (2.1)$$

where $\hat{A}_t$ is an advantage estimate (commonly computed with Generalised Advantage Estimation) [?, ?]. Intuitively, clipping limits the incentive to change the policy too aggressively, which helps prevent performance collapse from unstable updates—a practical consideration in contact-rich manipulation where learning can be sensitive to abrupt policy changes [?, ?].

2.1.0.5 Why PPO is often used for robotic manipulation

From a robotics perspective, Proximal Policy Optimization has several properties that make it attractive in manipulation pipelines:

- **Stable learning dynamics:** The clipped objective yields conservative and typically reliable updates [?].
- **Compatibility with continuous actions:** Proximal Policy Optimization naturally supports Gaussian policies over continuous controls [?].
- **Practical training loop:** The rollout-and-update structure integrates well with simulator-based training and supports systematic iteration on reward and termination design [?].

These considerations motivate Proximal Policy Optimization as a strong baseline for manipulation, especially when training stability and predictable iteration cycles are prioritized.

2.1.0.6 Implementation in this thesis

In this thesis, Proximal Policy Optimization is used as the primary learning algorithm, together with Generalised Advantage Estimation for advantage estimation, following standard actor–critic practice [?, ?]. The training loop is implemented with the `skrl` library, which provides modular Proximal Policy Optimization components (policy, value function, rollout storage, and optimization routines) and supports integration with simulator-based environments [?]. This allows the thesis to focus on environment formulation—observation design, action parameterization, reward shaping, termination conditions, and robustness

measures such as randomization—while relying on a consistent and reproducible Proximal Policy Optimization implementation [?].

2.2 Physics Simulation in Isaac Sim 5.1

Physics-based simulation in Isaac Sim 5.1 is implemented as a layered stack that separates (i) scene and asset description, (ii) binding of these descriptions to a concrete backend, and (iii) numerical solvers that evolve the physical state over time. In the Omniverse ecosystem, physics properties are authored in Universal Scene Description through the `UsdPhysics` schema (engine-agnostic physics description) and `PhysXSchema` (backend-specific extensions), allowing assets and environments to be defined declaratively in the scene graph [?, ?]. During simulation, the `omni.physx` runtime parses the Universal Scene Description stage, constructs the corresponding physics scene, advances the simulation, and writes back poses and state to Universal Scene Description [?]. Isaac Sim 5.1 uses NVIDIA PhysX 5 as its primary physics backend, providing rigid-body dynamics, articulations, contacts, and particle-based phenomena, with optional Graphics Processing Unit-accelerated pipelines for throughput-critical workloads such as robot learning experiments with many parallel environments [?, ?, ?].

A detailed understanding of the underlying physics models and numerical methods is required for two reasons. First, sim-to-real transfer is fundamentally limited by discrepancies between simulated and real dynamics (the *reality gap*), where unmodelled effects, incorrect parameters (e.g., friction and damping), and solver artefacts can lead to policies that exploit simulator-specific regularities [?, ?]. Second, in Reinforcement Learning, the simulator acts as a data-generating process; thus, modelling and solver choices directly shape the transition distribution used for learning, influencing both policy quality and generalization [?, ?]. In practice, this implies that solver selection, time-step choice, contact modelling, and collision geometry approximations must be treated as part of the learning system design, not as interchangeable implementation details [?, ?].

Placeholder for *Isaac Sim physics stack* (*USD Physics / PhysXSchema*
→ *omni.physx* → *PhysX 5 CPU/GPU solvers*) image

Figure 2.1: Conceptual physics simulation stack in Isaac Sim 5.1: physics is authored in Universal Scene Description via `UsdPhysics` and `PhysXSchema`, interpreted by `omni.physx`, and simulated using PhysX 5 with optional Graphics Processing Unit-accelerated pipelines.

2.2.1 Rigid Body Dynamics

Rigid-body simulation assumes that each object moves as a rigid transform, i.e., deformation is neglected and the state is represented by position/orientation and linear/angular velocity. PhysX represents the simulated world as a set of actors (static, kinematic, dynamic) with attached collision shapes and material properties; dynamics are advanced under external loads (e.g., gravity), contact constraints, and joint constraints [?, ?]. In Isaac Sim, these concepts are exposed through Universal Scene Description physics schemas and their PhysX extensions, enabling rigid bodies, colliders, and constraints to be authored on scene primitives [?, ?].

Articulated bodies, joints, and robot mechanisms. Robotic manipulators are modelled as articulated mechanisms, i.e., rigid links connected by joints that constrain relative motion and optionally apply actuation (drives). PhysX provides an *articulation* abstraction that is simulated in reduced coordinates, where the configuration is parameterized by the root pose and joint coordinates rather than the world pose of each link [?]. This formulation is designed to improve numerical fidelity for mechanisms (e.g., reduced joint error and improved behavior under large mass ratios) compared to independently simulating rigid bodies connected by generic constraints [?]. In this thesis context, the tendon-driven arm and the attached gripper are simulated as chains of rigid links with joint constraints and joint drives, such that control inputs act on joint coordinates while the solver enforces inter-link constraints and contacts [?, ?]. PhysX also exposes explicit *articulation tendons* as additional constraint structures within an articulation, including fixed tendons (coupling joint coordinates) and spatial tendons (spring-damper and limit constraints defined over attachment trees across links), which are relevant when cable-like actuation or joint coupling must be represented at the physics level [?, ?].

Contacts and friction models. Rigid-body collision response is formulated as a constrained dynamics problem in which contacts prevent interpenetration and can transfer or dissipate energy through restitution and friction [?, ?]. In PhysX, friction and restitution are specified via materials attached to shapes; friction is parameterized by static and dynamic coefficients and can be combined for contacting pairs using configurable combine modes [?, ?]. Because contact-rich manipulation is common in robotic assembly, grasping, and textile handling, parameter choices for friction and restitution are directly coupled to learning outcomes and to sim-to-real robustness [?, ?].

TGS constraint solving in PhysX. Rigid-body dynamics with contacts and joints is solved in PhysX using iterative constraint solvers. PhysX supports a classic PGS-style solver and the newer TGS solver, with TGS introduced in PhysX 5.1 and recommended for improved convergence and joint drive accuracy in many scenarios [?]. Both solvers operate iteratively over constraint manifolds (contacts, joints, friction), and accuracy is traded for performance by limiting iteration counts. A key motivation for temporal formulations is the observation that, for stiff constraint systems, replacing multiple solver iterations at a long time step by multiple smaller substeps can yield superior stability and accuracy at comparable computational cost [?, ?]. This perspective is consistent with PhysX guidance to consider smaller time steps (or substepping) when joint stability and contact behavior are insufficient under a fixed number of iterations, rather than indefinitely increasing iteration counts [?].

Simulation parameters and numerical tradeoffs. For robotics and Reinforcement Learning, the most consequential rigid-body parameters are those controlling temporal discretization, solver convergence, and contact generation. The simulation time step h determines how frequently dynamics are integrated and how often the policy interacts with the environment; overly large h can lead to missed collisions, unstable joint behavior, and biased contact impulses, whereas smaller h improves accuracy at increased compute cost [?, ?]. Solver iteration counts control how many passes are made over constraints; PhysX allows per-actor iteration counts (position and velocity iterations), and the effective iterations for a connected island are determined by the maximum requested within that island [?, ?]. Contact generation is influenced by the pair `contactOffset` and `restOffset`: contact points are generated once shapes are within the combined contact distance, while the rest offset defines the target separation distance at rest, enabling predictive enforcement and reducing jitter at the cost of introducing a small separation bias [?]. Finally, friction coefficients (static/dynamic) and restitution parameters are specified in materials and substantially affect sliding, sticking, and impact behavior [?, ?].

Table 2.1: Selected rigid-body simulation parameters relevant to robot learning in Isaac Sim / PhysX.

Parameter	Location	Primary effect
Time step h	Physics scene	Accuracy vs. throughput; contact/joint stability
Solver iterations	Actor / island	Constraint convergence; joint error; contact quality
<code>contactOffset</code> / <code>restOffset</code>	Shape	Contact prediction and resting separation
Friction coefficients	Material	Sliding/sticking behavior; grasp stability [?]

Graphics Processing Unit-accelerated rigid bodies for parallel Reinforcement Learning environments. PhysX provides a Graphics Processing Unit rigid-body pipeline that accelerates broad-phase, narrow-phase, contact generation, and constraint solving on CUDA-capable hardware, while exposing the same API concepts as the Central Processing Unit pipeline [?]. Large performance gains are expected when many rigid bodies are active, which aligns with robot learning workloads where hundreds to thousands of environment replicas are simulated in parallel to increase sample throughput [?, ?]. In IsaacLab, PhysX Graphics Processing Unit simulation is enabled by default and can be configured explicitly; however, Graphics Processing Unit simulation requires preallocation of internal buffers and may fail when insufficient capacities are specified, which makes sizing and profiling part of simulation configuration for large-scale training [?]. The broader motivation for Graphics Processing Unit-resident simulation in Reinforcement Learning is that physics stepping and policy learning can share device memory, avoiding Central Processing Unit–Graphics Processing Unit transfer bottlenecks; this design has been demonstrated to yield substantial throughput improvements in GPU-based robot learning platforms [?].

Collision detection and geometry approximations. Collision detection cost is highly sensitive to the chosen collider representation. Primitive shapes (e.g., spheres, boxes, capsules) provide the strongest performance and numerical robustness because intersection tests and contact generation are simple and well-conditioned [?]. Convex meshes provide a practical middle ground for complex objects by enabling efficient collision queries while approximating concave geometry; PhysX requires *convex mesh cooking* to transform input geometry into an optimized representation for collision detection [?]. In Isaac Sim, mesh colliders are often approximated by convex hulls by default for performance, with more expensive approximations available when needed [?]. When concavity is important for dynamic objects, convex decomposition (a set of convex parts) is typically preferred over a single triangle mesh collider because triangle-mesh contact generation is expensive and, depending on pipeline and version, may require additional structures. In PhysX 5.3+, dynamic triangle meshes in the Graphics Processing Unit collision pipeline require an associated Signed Distance Field to accelerate collision detection, increasing memory use but improving query performance [?]. Therefore, collider selection becomes a modelling choice that trades geometric fidelity against computational cost and solver stability, and this tradeoff must be balanced against the throughput requirements of Reinforcement Learning training.

2.2.2 Cloth and Deformable Object Simulation

Deformable object simulation relaxes the rigid-body assumption by allowing the object’s shape to change under load. This introduces two coupled complexities: the state dimension increases dramatically (many vertices/particles instead of a single rigid transform), and contacts become configuration-dependent because deformation continuously alters the set of potential contact points and normals [?, ?]. For textiles, this coupling is particularly pronounced: applied forces not only translate the object but also create folds, wrinkles, and contact-rich self-interactions that alter future dynamics. These properties are repeatedly identified as key contributors to the reality gap in robotic cloth manipulation [?].

Common modelling approaches. Three modelling families are most relevant in robotics simulation. First, mass–spring(-damper) models discretize cloth as a mesh whose vertices are connected by springs encoding stretch and (optionally) bending penalties; such models can be made stable with implicit integration techniques but require careful parameterization and can be numerically stiff [?]. Second, particle-based constraint methods represent cloth by particles and enforce desired behavior via geometric (holonomic) constraints, which can be solved iteratively with high robustness under collisions; Position-Based Dynamics and its variants are canonical examples [?, ?]. Third, continuum formulations discretize elastic energy and constitutive laws (e.g., in Finite Element Method) or use hybrid particle–grid methods (e.g., Material Point Method), providing a pathway to parameters with clearer physical interpretation at increased computational cost [?, ?].

Challenges specific to cloth dynamics. Cloth exhibits high effective Degree of Freedom, frequent self-collision, and strong sensitivity to material parameters such as bending stiffness, friction, and damping. Robust self-contact and friction handling is widely recognized as a hard requirement for stable cloth simulation, particularly under large deformations and repeated collisions [?, ?]. From a learning perspective, this sensitivity implies that small modelling errors (e.g., in friction or damping) can change grasp success rates, sliding behavior, and fold formation, which in turn can qualitatively alter the learned policy’s strategy and its transferability to hardware [?, ?].

2.2.2.1 PBD — PhysX Particle Cloth

PhysX provides particle-based simulation through `PxBBDParticleSystem`, whose solver is explicitly described as using Position-Based Dynamics for particle–particle dynamics

and as supporting phenomena including cloth, inflatables, and other deformable behaviors [?, ?]. In Omniverse, particle simulation is Graphics Processing Unit-accelerated and is documented as supporting fluids, granular media, and cloth, including interaction with rigid bodies and articulations [?]. At the feature level, particle cloth is treated as a particle-based Position-Based Dynamics mass–spring-style system and is marked as deprecated in favor of newer deformable-body features, which highlights that “cloth” in Isaac Sim can refer to multiple, evolving solver families rather than a single canonical model [?, ?].

Time-step structure and iterative projection. In standard Position-Based Dynamics, the deformable is represented by particle positions $\mathbf{x}_i \in \mathbb{R}^3$ with masses m_i ; desired behavior is enforced by a set of constraints $C_j(\mathbf{x})$ (equalities or inequalities), typically solved by iterative projection in a Gauss–Seidel fashion [?]. A schematic time step consists of: (i) updating velocities under external forces, (ii) predicting positions by an explicit step, (iii) iteratively projecting predicted positions to reduce constraint violations, and (iv) updating velocities from the corrected positions (and applying damping if configured) [?]. For a constraint $C_j(\mathbf{x}) = 0$, linearization yields a correction direction based on the constraint gradient. Using the inverse mass matrix $\mathbf{M}^{-1}$ (diagonal in the particle basis), one common update can be written as [?, ?]:

$$\Delta \mathbf{x} = k_j \cdot s_j \cdot \mathbf{M}^{-1} \bullet C_j(\mathbf{x}), \quad s_j = \frac{-C_j(\mathbf{x})}{\bullet C_j(\mathbf{x})^\top \mathbf{M}^{-1} \bullet C_j(\mathbf{x})}, \quad (2.2)$$

where $k_j \in [0, 1]$ is a user-defined stiffness-like scaling applied per constraint.

Effective stiffness and dependence on h and iteration count. A central practical implication of the iterative projection view is that the *effective* stiffness of constraints depends on numerical settings. Increasing the number of projection iterations typically yields stiffer constraint satisfaction at fixed h , while increasing h tends to reduce stability and can require either more iterations or smaller substeps to maintain comparable behavior [?, ?, ?]. This is beneficial for robustness and controllability, but it complicates mapping simulator parameters to physically measured material properties (e.g., bending stiffness in SI units), which is a recurring issue when calibrating cloth simulation for sim-to-real transfer [?, ?].

XPBD compliance formulation. Extended Position-Based Dynamics extends Position-Based Dynamics by introducing a compliance parameter and an explicit Lagrange multiplier state to reduce time-step dependence and recover meaningful constraint

force estimates [?]. For compliance $\alpha \geq 0$ and time step h , the incremental multiplier update for a constraint can be expressed as [?]:

$$\Delta\lambda = \frac{-C(\mathbf{x}) - \frac{\alpha}{h^2}\lambda}{\sum_i w_i \|\bullet_{\mathbf{x}_i} C(\mathbf{x})\|^2 + \frac{\alpha}{h^2}}, \quad \Delta\mathbf{x}_i = w_i \bullet_{\mathbf{x}_i} C(\mathbf{x}) \Delta\lambda, \quad (2.3)$$

with inverse masses $w_i = 1/m_i$. This formulation decouples the user parameter α from the iteration count in a way that improves interpretability of stiffness-like behavior across different h and solver settings [?].

Why Position-Based Dynamics is used in PhysX for particle cloth. Position-Based Dynamics is well-aligned with Graphics Processing Unit execution because constraints can be partitioned into independent sets and processed in parallel; graph coloring is a common strategy to schedule constraint batches without write conflicts on shared particles [?]. Omniverse documentation explicitly emphasizes Graphics Processing Unit-accelerated Position-Based Dynamics particles and their interaction with rigid bodies and articulations, which is essential for cloth manipulation scenes in robotics [?]. In addition, the PhysX particle system API is designed around a dedicated Position-Based Dynamics solver type (`PxPBDParticleSystem`), reinforcing that particle cloth is treated as a specific, robust, high-throughput model family within the PhysX multiphysics portfolio [?, ?].

2.2.2.2 VBD — Newton Thin Deformables

While PhysX particle cloth is based on explicit prediction and iterative position projection, implicit methods aim to improve stability under stiffness and large time steps by solving a time-discrete optimization problem. Vertex Block Descent is introduced as a block coordinate descent solver for the variational form of implicit Euler time integration, targeting convergence to the implicit Euler solution while maintaining unconditional stability under limited iteration budgets [?].

Variational implicit Euler and global energy minimization. Let $\mathbf{x}_t$ denote positions, $\mathbf{v}_t$ velocities, h the time step, and $\mathbf{M}$ the mass matrix. In the variational formulation, the next positions are obtained by minimizing a global energy that combines inertia and potential energy [?]:

$$\mathbf{x}_{t+1} = \arg \min_{\mathbf{x}} G(\mathbf{x}), \quad G(\mathbf{x}) = \frac{1}{2h^2} \|\mathbf{x} - \mathbf{y}\|_{\mathbf{M}}^2 + E(\mathbf{x}), \quad (2.4)$$

where $\mathbf{y} = \mathbf{x}_t + h\mathbf{v}_t + h^2\mathbf{a}_{\text{ext}}$ and $E(\mathbf{x})$ is the total potential energy. Velocities follow from the implicit Euler relationship $\mathbf{v}_{t+1} = (\mathbf{x}_{t+1} - \mathbf{x}_t)/h$ [?].

Vertex-level block coordinate descent and local Newton step. Vertex Block Descent solves the global problem by iterating over vertices and updating one vertex at a time while temporarily holding the others fixed, yielding a vertex-level Gauss–Seidel pattern with favorable parallelization via vertex coloring [?]. For vertex i , a local energy is minimized:

$$\mathbf{x}_i \leftarrow \arg \min_{\mathbf{x}_i} G_i(\mathbf{x}), \quad G_i(\mathbf{x}) = \frac{m_i}{2h^2} \|\mathbf{x}_i - \mathbf{y}_i\|^2 + \sum_{j \in F_i} E_j(\mathbf{x}), \quad (2.5)$$

where F_i is the set of force/energy elements incident to vertex i . Because G_i has only three degrees of freedom, a local Newton step is formed by solving a 3×3 linear system per vertex [?]:

$$\mathbf{H}_i \Delta \mathbf{x}_i = \mathbf{f}_i, \quad (2.6)$$

with $\mathbf{H}_i$ the local Hessian and $\mathbf{f}_i$ the local gradient/force term.

Advantages for stiff, contact-rich thin deformables. The primary advantages highlighted for Vertex Block Descent are: (i) unconditional stability inherited from the energy descent property, (ii) convergence toward the implicit Euler solution as iterations increase, and (iii) Graphics Processing Unit-oriented parallelization via vertex coloring with small, localized linear solves [?]. These properties are attractive for cloth-like thin deformables, which often exhibit near-inextensible stretch (high stiffness) combined with softer bending and frequent contact events; implicit/variational formulations can tolerate such stiffness ratios at larger time steps than explicit solvers with comparable iteration budgets [?].

Newton in the robotics simulation ecosystem. Newton is presented as an open-source, Graphics Processing Unit-accelerated physics engine designed for robotics simulation and robot learning, built on NVIDIA Warp and intended to integrate with robot learning frameworks such as IsaacLab [?, ?]. In NVIDIA’s robotics communication, Newton is explicitly associated with thin-deformable simulation via a Vertex Block Descent solver and with multiphysics capabilities that include implicit Material Point Method and other solver components, motivated by stability and performance for learning workflows [?, ?]. Accordingly, Vertex Block Descent is positioned as a solver choice when in-

creased stability under stiff materials and larger time steps is required, complementing the high-throughput Position-Based Dynamics-based particle cloth pathway in PhysX-centric stacks [?, ?].

2.2.2.3 Comparison: PBD vs. VBD for Cloth in Reinforcement Learning Workflows

Mathematical targets and implications. The core distinction is the optimization target. Position-Based Dynamics advances dynamics by predicting positions and then *projecting* them to reduce constraint violations, where the resulting behavior depends on solver scheduling and iteration count, and where constraint “stiffness” is typically not time-step invariant without extensions [?, ?]. Vertex Block Descent, in contrast, directly minimizes the variational implicit Euler energy and therefore targets a principled implicit integration objective; stability is maintained by construction via energy descent, and increased iteration budgets refine the approximation toward the implicit solution [?].

Practical tradeoffs for learning-centric simulation. For Reinforcement Learning, Position-Based Dynamics-based particle cloth is attractive because it is simple, robust under frequent contacts, and aligned with Graphics Processing Unit-parallel execution through constraint batching and coloring strategies; this supports the high environment throughput required for policy optimization [?, ?, ?]. However, because effective stiffness depends on h and iteration count, parameter tuning can be difficult to interpret physically and can amplify sim-to-real sensitivity if policies rely on artefacts (e.g., overly damped folds or under-resolved frictional sticking) [?, ?]. Vertex Block Descent is attractive when stability under stiffness, large time steps, and mixed constraint types is prioritized, particularly for systems that combine near-inextensible stretch with softer bending; such regimes are common in cloth manipulation and can be challenging for explicit projection methods at fixed compute budgets [?, ?].

Relation to the Omniverse ecosystem and thesis choice. In Omniverse/Isaac Sim, particle cloth is documented as deprecated in favor of newer deformable-body features, indicating an ongoing transition toward more general deformable schemas and alternative solver families [?, ?]. For the experiments in this thesis, the PhysX/Position-Based Dynamics particle cloth pathway is motivated by Graphics Processing Unit throughput and robustness for large-scale training in Isaac Sim 5.1, whereas Newton/Vertex Block Descent is conceptually aligned with scenarios where greater stability under stiff cloth behavior and larger time steps is required (e.g., when cloth parameters or contact conditions

become numerically challenging for projection-based solvers) [?, ?, ?].

2.3 NVIDIA Isaac Sim and IsaacLab Architecture

NVIDIA Isaac Sim (version 5.1) is used as the primary simulation environment in this thesis because it integrates robotics-oriented scene authoring, high-fidelity multi-physics simulation, and sensor-realistic rendering within a single Omniverse-based application stack [?]. In addition, the associated IsaacLab framework is used to structure the learning and evaluation workflow, because it provides a unified, modular interface for designing robotics tasks as Markov Decision Processes and training policies at scale using GPU-accelerated parallel simulation [?]. The resulting toolchain aims to minimize integration overhead between simulation fidelity (e.g., cloth–rigid interactions), robotic middleware interfaces, and the training pipeline used for robot learning.

2.3.1 Isaac Sim in the Omniverse application stack

Isaac Sim is implemented as a reference application on top of NVIDIA Omniverse and is designed to support development, simulation, and testing of AI-driven robots in physically based virtual environments [?]. A central architectural decision is the use of Universal Scene Description as the scene representation and interchange format: robotics assets and environments are expressed as a hierarchical scene graph with reusable primitives, enabling non-destructive composition and scalable instancing workflows [?]. This representation is used as the common “data layer” across physics, rendering, and robotics-specific tooling, which reduces interface mismatches between asset creation and simulation-time execution.

Isaac Sim is built on Omniverse Kit, which is a modular application framework based on lightweight plugins and an extension mechanism for composing functionality [?]. Consequently, Isaac Sim capabilities (e.g., physics interfaces, sensor simulation, and robotics middleware bridges) are exposed as extensions that can be enabled, configured, and scripted from Python, which supports both interactive development in the graphical application and headless execution in automated pipelines [?]. This extension-centric architecture is relevant to robot learning workloads because it enables the simulator to be configured as a reproducible experiment runtime, rather than as a monolithic interactive tool.

Placeholder for *layered architecture of Isaac Sim + IsaacLab* image

Figure 2.2: Schematic view of the software stack used in this thesis. Isaac Sim is built on Omniverse Kit with Universal Scene Description as the scene data layer and PhysX 5 as the physics backend; RTX-based rendering is used for sensor simulation. IsaacLab is implemented on top of Isaac Sim and provides task abstractions (e.g., manager-based Markov Decision Process environments) and training utilities (e.g., multi-Graphics Processing Units execution).

2.3.2 Core simulation features in Isaac Sim 5.1

Isaac Sim combines high-fidelity physics, photorealistic rendering, and robotics integration within a single runtime [?]. These features are not independent: physics fidelity affects contact-rich manipulation, rendering realism affects perception and sim-to-real transfer, and middleware integration affects how simulation is connected to external robot software stacks. The most relevant capabilities for this thesis are summarized below.

- **Omniverse and Universal Scene Description-centric scene representation:** Scenes are represented in Universal Scene Description, which serves as the unifying data interchange format for robot assets, environments, and simulation metadata [?]. This enables structured scene graphs and supports scalable instancing patterns for large simulation workloads.
- **High-fidelity physics based on PhysX 5:** Isaac Sim uses a GPU-based PhysX engine and provides access to rigid-body and articulated dynamics required for robotics simulation [?]. GPU rigid-body simulation in PhysX provides GPU-accelerated broad phase, contact generation, and constraint solving when enabled, while other components remain on the CPU [?].
- **Integrated deformables and cloth-relevant dynamics:** Omni PhysX exposes GPU-accelerated particle simulation based on Position-Based Dynamics, which supports effects including cloth-like particle systems and their interaction with articulations and other physics objects [?]. This is directly relevant for tasks where cloth dynamics and contact interactions must be represented in the same simulation runtime.
- **Photorealistic RTX-based sensor simulation:** Isaac Sim supports multi-sensor RTX rendering and uses GPU access to simulate sensors such as cameras and lidar in an industrially scalable manner [?]. RTX-based sensor simulation is exposed via dedicated extensions (e.g., RTX sensors) within the Kit extension framework [?].
- **Robotics middleware support via Robot Operating System 2 bridging:** A dedicated Robot Operating System 2 Bridge extension is provided to publish and subscribe to Robot Operating System 2 topics and services from within Isaac

Sim, enabling bidirectional communication between simulation and external Robot Operating System 2 systems [?].

2.3.2.1 GPU acceleration and parallel simulation

GPU acceleration in the Isaac ecosystem is achieved at multiple levels, ranging from physics backend execution to batched simulation of many environments. At the physics-backend level, PhysX GPU rigid bodies are configured at the scene level and enable GPU-accelerated rigid-body pipeline components including broad phase, contact generation, and the constraint solver [?]. Since not all features are executed on the GPU, the performance benefit depends strongly on scene composition and on whether large numbers of active bodies are simulated concurrently [?]. This architecture is aligned with robot learning workloads in which many similar environments are executed in parallel to increase sample throughput.

At the application level, Isaac Sim exposes explicit support for cloning and replicating environments. In particular, the `isaacsim.core.cloner` extension is provided to clone prims and environments efficiently and to optionally filter collisions across clones, which is a practical requirement when many environments share a single process runtime [?]. This cloning functionality forms a key bridge between a high-fidelity simulator and the throughput requirements of Reinforcement Learning, since scalable policy training is typically bottlenecked by the rate of environment rollouts rather than by single-scene fidelity.

2.3.3 IsaacLab as a unified robot learning framework

IsaacLab is used in this thesis as the primary framework for structuring robot learning experiments on top of Isaac Sim. IsaacLab is described as the successor to Isaac Gym and is intended to extend GPU-native simulation into large-scale, multi-modal robot learning, while remaining integrated with Isaac Sim for high-fidelity physics and RTX-based sensing [?]. The framework is designed to unify workflows spanning Reinforcement Learning, imitation learning, and motion planning, while exposing reusable abstractions for robot assets, environments, and training pipelines [?].

A key motivation for IsaacLab is the unification of two prior lines of development in the Isaac ecosystem. First, Isaac Gym demonstrated that physics simulation and policy learning can be executed entirely on the Graphics Processing Unit with direct tensor access, yielding large speedups over CPU-based simulators when training complex robot policies [?]. Second, the Orbit framework established a modular environment library on top of Isaac Sim, providing benchmark tasks (including cloth folding) and supporting policy

training and demonstration collection using GPU-based parallelization [?]. IsaacLab is positioned as a consolidated framework that carries forward the GPU-native paradigm of Isaac Gym and the modular task and environment design principles of Orbit, but within an Isaac Sim-integrated runtime that supports richer sensing and broader multi-physics capabilities [?].

2.3.3.1 Manager-based Markov Decision Process workflow and code reuse

IsaacLab structures simulation tasks explicitly as Markov Decision Processes, i.e., as environments that provide state (or observations), actions, and reward/termination signals suitable for learning. Within the manager-based workflow, a base environment is composed from reusable managers (e.g., action, observation, and event managers) and is then extended to an Reinforcement Learning task environment by adding reward, termination, command, and curriculum components [?]. This decomposition is relevant for thesis-scale experimentation because it separates environment implementation from task specification: instead of modifying core environment classes directly, task-specific behavior is encoded in configuration objects, which enables consistent reuse across tasks and reduces duplication across experimental variants [?].

The manager-based formulation also encourages a stable interface between simulation and training. In the documented workflow, the `ManagerBasedRLEnv` interface returns Markov Decision Process-relevant signals (including reward and termination status) while remaining compatible with standard environment creation patterns (e.g., via `gymnasium.make`), which supports integration with multiple Reinforcement Learning libraries without rewriting the simulator interface layer [?]. The resulting abstraction is intended to enable systematic experimentation across robot morphologies and task configurations while preserving a consistent training loop.

2.3.3.2 Distributed training support in IsaacLab

Robot learning in high-dimensional, contact-rich tasks often requires a large number of environment interactions, which motivates distributed execution. IsaacLab provides explicit support for multi-Graphics Processing Unit and multi-node Reinforcement Learning training, with documented workflows using PyTorch distributed tooling (e.g., `torchrun`) and a process-per-Graphics Processing Unit execution model in which each process instantiates its own IsaacLab environment and synchronizes gradients across processes [?]. This design is aligned with on-policy Reinforcement Learning settings, in which large batch sizes and parallel rollouts can substantially increase training throughput, provided

that simulator stepping and observation extraction can be executed efficiently.

The documented multi-Graphics Processing Unit pipeline is also operationally relevant: each process is responsible for its own environment instance and rollout collection buffers, while synchronization occurs only during gradient updates [?]. This reduces coupling between simulation instances and avoids centralized simulation bottlenecks, at the expense of higher per-process resource usage. For thesis experiments that require extensive policy iteration or hyperparameter exploration, this design provides a practical path toward scaling without abandoning the integrated Isaac Sim runtime.

2.3.3.3 Imitation learning and motion planning integration

Beyond Reinforcement Learning, IsaacLab supports imitation learning workflows that combine human demonstration segments with automated planning. The SkillGen pipeline is documented as integrating motion planning into the IsaacLab Mimic demonstration workflow, thereby generating collision-free trajectories between skill segments using an external planner [?]. In this workflow, cuRobo is used as the underlying motion planner for generating smooth, collision-free motions, and the planner is explicitly described as GPU-accelerated [?].

cuRobo itself is formulated as a GPU-parallel motion generation approach for manipulators, with a design that exploits massive parallelism (e.g., many optimization seeds and parallel collision checks) to achieve low-latency planning and inverse kinematics computations [?]. The availability of such a planner within the IsaacLab ecosystem is relevant for tasks where expert trajectories must be generated efficiently, either to bootstrap learning (e.g., as demonstrations) or to define hybrid learning-and-planning pipelines.

2.3.4 Rationale for selecting Isaac Sim 5.1 with IsaacLab

The combined selection of Isaac Sim 5.1 and IsaacLab is driven by the need to simultaneously address (i) deformable/cloth dynamics, (ii) contact-rich manipulation in articulated robots, and (iii) scalable learning infrastructure. These requirements are tightly coupled in the targeted thesis use case: cloth manipulation requires reliable coupling between deformable dynamics and rigid contacts, while learning-based policies require large numbers of rollouts to converge.

Integrated cloth and deformable dynamics are supported within the Isaac Sim / PhysX stack through GPU-accelerated particle-based simulation that can represent cloth-like behavior and interact with articulations and other physics objects within a shared simulation scene [?]. In addition, IsaacLab explicitly highlights support for deformable objects

(including cloth) within a framework built on Isaac Sim and designed for multi-modal learning, indicating that deformable interactions are treated as a first-class concern in the simulation-to-learning pipeline [?]. This integration is beneficial because it avoids ad-hoc coupling between an external deformable simulator and a separate rigid-body engine, which would otherwise require custom synchronization, contact handling, and state transfer between solvers.

Scalability is addressed through GPU-native simulation and distributed training utilities. Isaac Gym established that GPU-resident simulation and learning with direct tensor access can yield large improvements in end-to-end training throughput compared to CPU-based simulation paired with GPU-based neural network training [?]. IsaacLab extends this paradigm within an Isaac Sim-integrated stack and provides documented support for multi-Graphics Processing Unit and multi-node training, enabling scale-out execution for Reinforcement Learning workloads when training is bottlenecked by rollout collection [?, ?]. When combined with Isaac Sim cloning utilities for replicated environments [?], the simulator can be configured to support large-scale parallel rollouts within a consistent runtime, which is particularly advantageous in on-policy scenarios where sample throughput dominates wall-clock training time.

Finally, robotics integration requirements are addressed at the middleware level. Isaac Sim exposes a Robot Operating System 2 Bridge extension for publishing and subscribing to Robot Operating System 2 topics and services through OmniGraph nodes and action graphs [?]. This bridge supports a common development pattern in which simulation is used for software-in-the-loop or hardware-in-the-loop testing, and it provides a practical interface to connect simulation-time controllers and perception to external robot software stacks without bespoke transport layers.

2.3.5 Comparison to alternative simulation platforms

The choice of simulator is constrained by trade-offs between fidelity, performance, and integration effort. For this thesis, the critical differentiator is the ability to combine contact-rich articulated manipulation with cloth dynamics and scalable learning infrastructure within a unified and scriptable runtime. Alternative platforms are briefly contrasted below to justify the selection of Isaac Sim 5.1 with IsaacLab for this specific use case.

MuJoCo is a widely used physics engine for model-based control that emphasizes efficient dynamics and contact simulation, and it is commonly adopted as a baseline simulator in continuous-control research [?]. Its design makes it well-suited for rigid-body control benchmarks and for iterative controller development when the primary bottleneck is per-step simulation performance. However, the thesis use case requires integrated cloth dy-

namics and sensor-realistic rendering within the same architecture, and these requirements typically necessitate additional tooling beyond a core rigid-body simulator.

PyBullet is widely used as a Python-accessible simulation interface built around the Bullet physics engine, and it is often adopted in robotics prototyping due to its ease of scripting and broad ecosystem integration [?]. The underlying Bullet engine is documented in the context of real-time physics simulation and is frequently used in interactive applications [?]. In practice, PyBullet is frequently selected when rapid iteration and lightweight deployment are prioritized, but the simulator stack used in this thesis additionally requires an integrated high-fidelity rendering and sensor simulation pipeline and a unified large-scale training framework, which are provided natively by Isaac Sim and IsaacLab rather than assembled from separate components.

SOFA is an open-source simulation framework primarily targeted at medical simulation and is designed around an advanced software architecture for building complex deformable simulations by composing algorithms, solvers, and collision models [?]. This makes SOFA a strong candidate for high-fidelity deformable modeling, including scenarios where accurate soft-tissue mechanics are required. Nevertheless, the overall thesis workflow requires not only deformable simulation but also scalable robot learning infrastructure with high-throughput parallel environment execution and integrated robotics middleware interfaces, which are not the primary design objectives of SOFA and typically require additional integration layers.

Table 2.2: Qualitative comparison of simulators with respect to thesis requirements.

Platform	Primary strength (per source)
Isaac Sim + IsaacLab	Integrated Omniverse stack with PhysX, RTX, and Reinforcement Learning [?]
MuJoCo	Efficient physics engine for model-based control [?]
PyBullet/Bullet	Accessible real-time physics simulation ecosystem [?, ?]
SOFA	Modular framework for complex deformable simulation [?]

Overall, Isaac Sim 5.1 with IsaacLab is selected because it provides an integrated software architecture that aligns with the requirements of cloth manipulation and scalable robot learning. The Isaac stack combines a Universal Scene Description-based scene representation, PhysX-backed multi-physics simulation, RTX-based sensing, and robotics middleware integration in a single Omniverse runtime [?, ?]. IsaacLab further provides a principled Markov Decision Process-based task design workflow and documented multi-Graphics Processing Unit/multi-node training support, which reduces the engineering effort required to perform large-scale learning experiments while retaining simulation fi-

delity [?, ?, ?].

2.4 Tendon-Driven Robot Kinematics and Modeling

Tendon-driven robots are articulated mechanisms in which joint motion is generated by tensile elements (tendons) that transmit forces from remote actuators to the joints through pulleys, guides, or routing channels. In this actuation paradigm, the actuators and transmissions can be placed proximally, while the distal structure remains lightweight. This design is widely used in continuum and cable-driven mechanisms, particularly when low distal inertia, compact joint packaging, or intrinsic compliance is desired [?, ?]. At the same time, the mechanical transmission characteristics of tendons introduce additional modeling complexity. The mapping between actuator commands and joint motion depends on geometry (routing and moment arms), tendon elasticity, and non-idealities such as friction and hysteresis, which are frequently reported as dominant error sources in cable- and tendon-driven systems [?, ?].

2.4.1 Tendon-driven actuation principles

In tendon-driven architectures, actuation is typically realized by commanding tendon displacement (spooling length) or tendon tension. For common antagonistic arrangements, a joint is actuated by at least two tendons such that differential tension generates a net joint torque while co-contraction modifies apparent stiffness. When tendons span multiple joints, a single tendon displacement can induce coupled motions, which is advantageous for compact designs but complicates decoupled joint control [?]. The mechanical benefit of remote actuation is that motors and gear reductions can be located near the base, reducing distal mass and inertia and thereby enabling higher angular accelerations and dynamic performance for a given torque budget [?].

The same properties that reduce distal inertia also reduce transmission rigidity. Tendon stretch, cable-pulley compliance, and frictional effects can cause configuration-dependent motion errors, phase lags, and hysteresis loops under load. These effects are not only relevant for force control but also for kinematic accuracy because they break the assumption of a unique, memoryless mapping between actuator displacement and end-effector pose [?, ?]. As a result, tendon-driven systems are often controlled using either (i) local joint sensing (e.g., encoders placed near joints) to close the loop in joint space, or (ii) model-based compensation and identification to improve the actuator-to-joint mapping, depending on the available sensing and the targeted accuracy [?].

2.4.2 Kinematic modeling: from tendon space to task space

Rigid-link jointed mechanisms. For tendon-driven mechanisms built from rigid links and discrete joints, the kinematic structure in task space is formally identical to that of a direct-drive serial manipulator once the joint coordinates are defined. The principal difference is that the commanded variables are naturally expressed in tendon space (tendon lengths or tensions) rather than joint space. Consequently, the overall mapping is typically decomposed into two stages: (i) a transmission model from tendon variables to generalized joint coordinates, and (ii) a conventional rigid-body forward kinematic chain from joint coordinates to end-effector pose. In practice, the transmission model is required because routing geometry and tendon elasticity determine how a commanded spool displacement converts into a joint angle, and multi-joint tendons can create coupling terms that are absent in direct-drive designs [?].

Continuum and cable-driven bending mechanisms. For tendon-driven continuum robots, joint coordinates are not associated with discrete revolute joints but with distributed deformation of a flexible backbone. A widespread modeling approach is the piecewise constant-curvature assumption, in which the backbone is approximated as a sequence of constant-curvature arcs. This assumption reduces an effectively infinite-dimensional shape description to a finite set of arc parameters, enabling closed-form forward and differential kinematics for many designs [?]. The assumption is frequently used as a baseline model and remains attractive for control and planning because it yields tractable Jacobians and efficient computations. However, tendon friction and hysteresis can substantially degrade the predictive accuracy of purely geometric constant-curvature mappings, motivating extended models that incorporate frictional memory effects and tension propagation along the tendons [?].

Implications for inverse kinematics and control. In tendon-driven systems, the Inverse Kinematics problem is frequently formulated in actuator space (tendon lengths) rather than in joint space. Even if a rigid-link Inverse Kinematics solution exists in terms of joint angles, the implementation of that solution on real tendon-driven hardware can be limited by transmission nonlinearities. In particular, hysteresis implies that the same tendon command can yield different end-effector poses depending on motion history and external loading, such that a purely kinematic Inverse Kinematics solution may not be repeatable without feedback [?]. For learning-based control pipelines, this motivates the separation between an *idealized kinematic model* used for planning or workspace analysis and a *simulation/control model* that includes compliance and control bandwidth limita-

tions to represent actuation uncertainty [?].

2.4.3 Robot configuration used in this project

The robotic system considered in this thesis combines a virtual positioning stage with a tendon-driven arm and a parallel-jaw gripper. The complete assembly is modeled as a serial chain in which a planar repositioning stage provides coarse positioning of the arm base, and a tendon-driven wristed arm provides dexterous local manipulation. The design builds on the tendon-driven robot concept and mechanical arm design described by Klein [?].

Kinematic structure. The manipulator is composed of three modules: (i) a 2-Degree of Freedom linear stage modeled as two orthogonal prismatic joints, (ii) a 3-Degree of Freedom tendon-driven arm modeled as three revolute joints, and (iii) a two-finger gripper attached at the wrist flange for grasping. For workspace and dexterity measures that depend on end-effector pose, the gripper opening is not treated as a kinematic Degree of Freedom affecting pose (it affects grasp state rather than tool pose) and is therefore excluded from the pose workspace computation.

A minimal generalized coordinate vector for pose-related analysis is defined as

$$\mathbf{q} = \begin{bmatrix} q_y & q_z & q_1 & q_2 & q_3 \end{bmatrix}^T,$$

where q_y and q_z denote the prismatic stage displacements and q_1, q_2, q_3 denote the arm revolute joint angles (e.g., one proximal bending joint and two wrist tilt axes). The joint limits bound the feasible configuration space and are imposed explicitly in the workspace analysis in subsection 2.4.6.

Placeholder for *robot kinematic chain schematic* image

Figure 2.3: Schematic of the kinematic chain used for modeling and workspace analysis: two orthogonal prismatic joints for base repositioning, followed by a three-revolute-joint tendon-driven arm and an end-effector frame located at the gripper mounting interface.

Rationale for the modular decomposition. The stage–arm decomposition separates coarse positioning from local dexterity. From a modeling viewpoint, this separation is advantageous because it allows the tendon-driven arm (which is sensitive to transmission compliance and cable routing) to be analyzed both with and without the additional stage

Table 2.3: Pose-relevant degrees of freedom for workspace analysis. Joint limit values are defined by the robot specification and are applied during sampling.

Joint	Type	Joint limits
q_y (stage)	prismatic	$[q_{y,\min}, q_{y,\max}]$
q_z (stage)	prismatic	$[q_{z,\min}, q_{z,\max}]$
q_1 (arm)	revolute	$[q_{1,\min}, q_{1,\max}]$
q_2 (arm)	revolute	$[q_{2,\min}, q_{2,\max}]$
q_3 (arm)	revolute	$[q_{3,\min}, q_{3,\max}]$

Degree of FreedomDegrees of Freedom. In addition, it supports environment design and task placement by making the reachable workspace tunable through stage limits without changing the arm geometry [?].

2.4.4 Simulation considerations for tendon-driven systems in Isaac Sim

Tendon-driven mechanisms pose specific simulation challenges because their behavior depends on cable routing, compliance, and frictional effects that are not captured by joint kinematics alone. In Universal Scene Description-based simulation pipelines, these effects must either be represented explicitly using tendon/cable primitives or approximated through coupled joints and compliant actuation.

Articulation tendons and coupled Degree of FreedomDegrees of Freedom. Omniverse Physics (PhysX backend) provides an articulation tendon concept with two main variants: fixed tendons (coupling joint coordinates) and spatial tendons (imposing constraints on distances between selected links through attachment points). These constructs are intended to represent coupled joint relationships and cable-like constraints inside articulated mechanisms [?]. In principle, joint coupling through fixed tendons can emulate certain tendon-driven topologies by enforcing linear relationships between generalized coordinates. Spatial tendons can represent a cable path between attachments but still abstract away detailed contact mechanics inside routing guides [?].

URDF mimic joints as a pragmatic approximation. When a robot is imported via Unified Robot Description Format, coupled motion can be encoded by mimic joints. The Isaac Sim Unified Robot Description Format importer can apply the PhysX mimic

joint mechanism to joints carrying a mimic tag, enabling linear coupling between a reference joint and a mimicking joint [?]. This mechanism is frequently used to represent mechanically linked joints such as parallelogram linkages or differential couplings. For tendon-driven approximations, mimic joints can be used to enforce idealized coupling relationships that would otherwise arise from shared tendon routing. It is noted in the Omni Physics documentation that mimic joints can introduce numerical instability when enforced as stiff constraints and that compliance can be introduced to improve stability when competing constraints apply [?].

MJCF tendon elements. MuJoCo XML Format models (MuJoCo XML) include tendon elements to express fixed and spatial tendons explicitly. The Isaac Sim MuJoCo XML Format importer exposes a tendon element for creating cable-like constraints that can span multiple bodies and joints, including fixed and spatial tendon types [?]. This provides an additional modeling route for tendon-driven mechanisms when an MuJoCo XML Format description is available or when it is convenient to prototype tendon constraints in MuJoCo-style syntax before conversion to Universal Scene Description.

Limitations relevant to tendon-driven fidelity. Although tendon primitives and mimic couplings allow tendon-like behavior to be introduced into an articulation, several fidelity gaps remain for tendon-driven robots: (i) distributed friction along tendon routing and hysteresis are typically not captured by ideal constraints, (ii) tendon stretch and tension propagation require tuned compliance parameters to remain stable at simulation time steps, and (iii) multi-contact routing through pulleys and guides is usually simplified to a small set of attachment points. These limitations are significant because tendon friction and hysteresis are reported to dominate motion errors in tendon-driven systems in applications such as surgical robotics, and improved prediction accuracy is obtained only when frictional effects are included in the mapping [?, ?]. As a consequence, tendon-driven robots are often simulated using a hybrid strategy: the *kinematic chain* is represented by standard joints, while tendon effects are represented by coupled joints (tendons or mimic constraints) and by compliant joint drives tuned to reproduce the apparent stiffness/damping observed in the physical system. Where higher fidelity is required, custom actuation models that map tendon commands to joint torques with friction/hysteresis compensation are required [?].

2.4.5 Precision, dynamics, and compliance: implications for cloth manipulation

Tendon-driven designs differ from direct-drive manipulators in both achievable dynamics and precision characteristics. Reduced distal inertia can enable rapid wrist motions and high dynamic bandwidth in configurations that would be limited by motor mass and gear-box inertia if actuators were placed distally [?]. However, because the actuation forces are transmitted through compliant tendons and routed interfaces, the closed-loop behavior is sensitive to friction, elastic stretch, and hysteresis effects. These phenomena can manifest as configuration-dependent tracking error and reduced repeatability compared to stiff, direct-drive solutions, and they motivate calibration and model-based compensation strategies when high positional accuracy is required [?, ?].

For cloth interaction tasks, the same compliance mechanisms that degrade open-loop positioning precision can be advantageous. Cloth manipulation is characterized by frequent contact, uncertain contact locations, and large modeling error due to deformable object dynamics. In this context, compliance can reduce peak contact forces, mitigate damage to the environment, and tolerate contact uncertainty by allowing small deviations from commanded trajectories without large impulsive forces [?, ?]. The safety-performance tradeoff in physical interaction is classically addressed by introducing compliance in the actuation/transmission, which reduces impact severity and can improve robustness in contact-rich tasks at the cost of reduced rigidity [?]. Therefore, tendon-driven designs can be regarded as mechanically aligned with manipulation scenarios in which force limiting and safe contact are prioritized over sub-millimeter absolute accuracy.

2.4.6 Workspace analysis and dexterity metrics

The workspace of a serial manipulator is the set of all end-effector poses that are reachable under joint limits and kinematic constraints. Workspace analysis is used to verify that a robot can reach task-relevant regions and to identify configurations that should be avoided due to singularities or poor motion quality [?, ?].

Forward kinematics and sampling motivation. Forward kinematics (Forward Kinematics) is defined as the mapping from a joint configuration $\mathbf{q}$ to an end-effector pose, i.e., position and orientation of a designated tool frame. For a general serial chain, this mapping can be written as

$$\mathbf{T}_{EE} = f(\mathbf{q}), \tag{2.7}$$

where $\mathbf{T}_{\text{EE}} \in SE(3)$ denotes a homogeneous transformation of the end-effector frame. In many practical systems, analytical derivation of workspace boundary surfaces is intractable due to high Degree of Freedom, joint coupling, or non-standard kinematics. In such cases, the workspace is frequently approximated numerically by sampling joint space and evaluating Forward Kinematics repeatedly. The Monte Carlo approach is widely used because it is simple to implement, directly incorporates joint limits, and does not require explicit boundary derivation [?].

Monte Carlo workspace sampling and voxel aggregation. In the Monte Carlo approach, configurations are sampled from a prescribed distribution over the admissible joint ranges. In the simplest implementation, each joint q_i is sampled independently and uniformly within its limit interval. For each sample $\mathbf{q}^{(k)}$, the corresponding end-effector pose is computed by Forward Kinematics. The resulting set of poses is then aggregated into a discrete spatial representation, e.g., Cartesian voxels, to estimate reachability statistics per region. This computation is conceptually aligned with established robotics software workflows that generate reachable workspace point sets paired with joint configurations [?].

When voxel aggregation is used, the workspace is partitioned into a regular grid of bins (voxels) in position space (and optionally orientation space). Each sample contributes to the occupancy count of the voxel containing the sampled pose. This discretization supports additional metrics beyond mere reachability, including density-based indicators of how many joint-space configurations map to a given region [?].

Placeholder for *workspace voxelization and density visualization* image

Figure 2.4: Example visualization of a sampled reachable workspace. End-effector positions obtained from Monte Carlo Forward Kinematics sampling are aggregated into Cartesian voxels. Voxel occupancy represents reachability density, and additional overlays (e.g., manipulability) can be displayed to indicate dexterity variation across task space.

Yoshikawa manipulability. Dexterity is commonly evaluated using Jacobian-based indices. Let $J(\mathbf{q})$ denote the geometric Jacobian that maps joint velocities to end-effector twist. The Yoshikawa manipulability measure is defined as [?]

$$w(\mathbf{q}) = \sqrt{\det(J(\mathbf{q})J(\mathbf{q})^\top)}. \quad (2.8)$$

This scalar is proportional to the volume of the manipulability ellipsoid and becomes zero at kinematic singularities where the Jacobian loses rank. Higher values indicate that

small joint motions can produce motion in a wider range of Cartesian directions, which is desirable for tasks requiring dexterous repositioning and local adjustments [?].

Inverse condition number as an isotropy index. A complementary measure is derived from the Jacobian condition number. Salisbury and Craig introduced the use of Jacobian conditioning to identify well-conditioned (isotropic) operating points and to quantify error amplification through the kinematic mapping [?]. Let $\sigma_{\min}$ and $\sigma_{\max}$ denote the minimum and maximum singular values of $J(\mathbf{q})$. The inverse condition number is defined as

$$\kappa_{\text{inv}}(\mathbf{q}) = \frac{\sigma_{\min}(J(\mathbf{q}))}{\sigma_{\max}(J(\mathbf{q}))}. \quad (2.9)$$

This index lies in $[0, 1]$, approaches zero near singular configurations, and equals one at perfectly isotropic configurations (equal singular values), where motion/force transmission properties are directionally uniform [?].

Reachability density. Reachability density is defined here as the number of sampled configurations whose end-effector poses fall within a given voxel. Under uniform joint sampling, this count is proportional to the joint-space volume that maps into the voxel region; thus, higher density indicates that more distinct configurations can realize poses in that region. This concept is closely related to workspace density formulations that assign each voxel the number of reachable points (or a normalized probability density) and interpret higher density as higher positional/orientational reach richness and, in discretely actuated systems, higher achievable pose resolution [?].

Implications for task-space design in learning-based manipulation. For manipulation tasks in cluttered scenes and for Reinforcement Learning-based control, the workspace and dexterity metrics directly inform environment layout and action-space design. Task objects (e.g., pick targets, cloth placement zones) are preferably positioned in regions with high reachability density and consistently high manipulability to reduce the probability that the robot is driven into near-singular configurations during exploration. Similarly, action bounds for end-effector motion primitives can be restricted to dexterous regions to improve learning stability and to reduce large corrective motions caused by kinematic ill-conditioning [?, ?, ?]. When tendon-driven compliance is present, these considerations become more important because poor kinematic conditioning can amplify transmission uncertainties into larger Cartesian errors, which in turn increases undesired contact forces during cloth interaction [?, ?].

2.5 Policy Learning Pipeline

Reinforcement learning-based control is typically realized as an iterative interaction loop between an agent (policy) and an environment (simulation), in which behavior is improved by maximizing an expected return defined over episodes [?, ?]. In IsaacLab, task construction is formalized through an explicit Markov Decision Process specification and a software architecture that decomposes environment logic into reusable terms for observations, actions, rewards, terminations, events, and curriculum scheduling [?, ?, ?]. This section describes the standard Reinforcement Learning pipeline under this workflow and derives an Markov Decision Process formulation suitable for cloth grasping and stretching tasks in simulation.

2.5.1 Agent–Environment Cycle and Markov Decision Processes

An Markov Decision Process is commonly defined as a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ denotes the action space, $\mathcal{P}(s' | s, a)$ denotes the transition kernel, $r(s, a, s')$ denotes the immediate reward, and $\gamma \in [0, 1)$ denotes the discount factor [?, ?]. The Reinforcement Learning objective is formulated as maximizing the expected discounted return under a policy π :

$$J(\pi) = \mathbb{E} \left[\sum_{t=0}^{T-1} \gamma^t r(s_t, a_t, s_{t+1}) \right], \quad a_t \sim \pi(\cdot | o_t), \quad s_{t+1} \sim \mathcal{P}(\cdot | s_t, a_t), \quad (2.10)$$

where T denotes the episode horizon and o_t denotes the agent’s observation at time step t [?, ?].

For robotic manipulation tasks, especially those involving deformable objects, the *true* state s_t (e.g., all cloth vertex/particle states and contact states) is typically not directly observable, and the policy is conditioned on a partial observation $o_t = g(s_t)$ constructed from sensors or engineered features [?, ?]. This setting is naturally described as a partially observable Markov Decision Process (Partially Observable Markov Decision Process), in which observations are related probabilistically to the hidden state and optimal decision making may require memory or state estimation [?]. In practice, many manipulation policies are implemented as *memoryless* mappings $\pi(a | o)$, such that the observation vector (or a short history of observations) is treated as the effective policy input [?, ?].

The standard on-policy learning pipeline alternates between (i) collecting rollouts by executing the current policy in the environment and (ii) updating the policy parameters by

optimizing a surrogate objective using the collected trajectories [?]. For example, Proximal Policy Optimization explicitly follows this alternation and performs multiple epochs of minibatch optimization over a clipped surrogate objective to stabilize policy gradient updates [?]. Regardless of the specific algorithm, this structure implies that the environment must provide: batched observations o_t , the ability to apply actions a_t , a reward signal r_t , and episode termination flags to delineate trajectories for return/advantage estimation [?, ?].

2.5.2 IsaacLab MDP Workflow for Task Construction

IsaacLab provides two task design workflows: a *manager-based* workflow, in which environment functionality is decomposed into modular managers, and a *direct* workflow, in which environment logic is implemented in a single class in a more monolithic style [?, ?]. The manager-based workflow is explicitly organized around Markov Decision Process components such as observations, actions, rewards, terminations, and curriculum, where each component is represented by configuration classes (implemented with a Python `@configclass` pattern) and is coordinated by the environment instance [?, ?, ?].

Placeholder for *diagram of IsaacLab manager-based MDP workflow:*
observation/action/reward/termination/curriculum managers
coordinated by env.step() image

Figure 2.5: Conceptual decomposition of an IsaacLab Markov Decision Process into managers. In the manager-based workflow, action processing, observation computation, reward accumulation, and termination checks are implemented as modular terms and orchestrated by the environment step loop.

In the manager-based workflow, IsaacLab defines reusable *term functions* (e.g., joint-state observations, generic reward primitives, and standard termination checks) in a dedicated Markov Decision Process module, which is designed to be consumed by corresponding managers [?, ?]. Concretely, observation terms are configured via an observation manager, action terms via an action manager, reward terms via a reward manager, and termination terms via a termination manager [?, ?]. Additionally, an event manager is typically used for randomization and episodic events, and a curriculum manager is provided to schedule changes to environment parameters or term weights over training [?, ?].

The direct workflow is supported for cases where decomposition is inconvenient or where large blocks of logic are implemented in optimized kernels, but it retains the same high-level requirements imposed by Reinforcement Learning: an environment class must provide observations, apply actions, compute rewards, and implement reset/termination logic [?,

?). Both workflows are compatible with ecosystem interfaces influenced by the OpenAI Gym API, which standardizes methods such as `reset()` and `step()` for Reinforcement Learning benchmarking and algorithm development [?, ?].

2.5.3 MDP Definition for Cloth Manipulation

The cloth manipulation task considered in this thesis is characterized by (i) high-dimensional deformable-object dynamics, (ii) contact-rich interactions during grasping, and (iii) task objectives that are naturally expressed through geometric measures such as cloth spread/coverage, corner-to-corner distances, and alignment with a target configuration [?, ?, ?]. While the *true* simulator state may include all internal cloth degrees of freedom, the Markov Decision Process induced for learning is typically specified through an observation function, an action parameterization, a shaped reward, and explicit termination conditions.

Table 2.4: Recommended Markov Decision Process specification elements for cloth grasp-and-stretch in simulation.

Component	Design choice	Rationale
Observations o_t	Proprioception + cloth features / vision	Partial observability and h
Actions a_t	Joint targets or end-effector deltas + gripper	Continuous control with c
Reward r_t	Coverage/stretching + grasp success + penalties	Dense shaping for spars
Termination	Timeout + success + safety/failure events	Stable episodes and sa

2.5.3.1 State and observations

Underlying state. In an Markov Decision Process formulation, the state $s_t \in \mathcal{S}$ is assumed to satisfy the Markov property, i.e., s_{t+1} is conditionally independent of past states given (s_t, a_t) [?, ?]. For deformable-object manipulation, s_t may be conceptualized as a concatenation of robot states (joint positions/velocities, tendon states, actuator internal states) and cloth states (e.g., all cloth vertex/particle positions and velocities, plus contact and constraint states) [?, ?]. This state is generally too high-dimensional to use directly for learning on real hardware, and it is often not directly measurable even in simulation when the goal is to preserve a realistic sensing setup [?, ?].

Observation design. The policy input is therefore defined as an observation vector $o_t \in \mathcal{O}$ constructed from simulated sensors and engineered features. In cloth-like de-

formable object tasks, a central difficulty is that sufficient low-dimensional state representations are hard to define and that observation is typically partial or noisy, motivating Partially Observable Markov Decision Process modelling assumptions [?, ?]. In simulation, observations may be selected along a spectrum from privileged (full cloth state) to realistic (vision/tactile-based). The following observation groups are commonly used for cloth manipulation and align with IsaacLab’s modular observation manager concept [?, ?, ?, ?]:

- **Robot proprioception:** Joint positions/velocities, end-effector pose, gripper opening, and optionally actuator-related signals (e.g., efforts) to support closed-loop manipulation [?, ?].
- **Cloth configuration features:** Reduced descriptors such as cloth corner positions, edge/contour points, keypoint sets, or a heightmap/coverage estimate, reflecting the observation biases reported in cloth-shaping systems that track corners and contours [?].
- **Visual observations:** RGB, depth, or RGB-D camera inputs. Image-based policies have been trained to infer grasp points and pull vectors for fabric smoothing, and depth has been shown to provide valuable information in such settings [?]. Benchmark studies also emphasize that learning purely from visual observations can be substantially more challenging than learning from ground-truth state in deformable tasks [?].
- **Contact and event signals:** Binary/continuous indicators derived from contact sensing (e.g., cloth grasp detected, contact forces above threshold) to disambiguate grasp success and failure events under partial observability [?].

In IsaacLab manager-based environments, these observation components are operationalized by defining observation terms and grouping them into observation groups, which are then evaluated each environment step to produce the policy input tensor [?, ?].

2.5.3.2 Action space

Low-level versus higher-level actions. For robotic manipulation, the action space $\mathcal{A}$ can be defined at different abstraction levels. At the lowest level, actions may be joint torques $\boldsymbol{\tau} \in \mathbb{R}^n$ applied to an n -Degree of Freedom manipulator, corresponding to direct actuation in the robot dynamics model [?]. Alternatively, actions may be expressed as joint velocity commands, joint position targets, or incremental joint displacements, which are then realized through an internal controller in the simulator or robot stack [?]. In deformable manipulation, action abstraction is frequently used to reduce exploration complexity and to stabilize contact interactions, since learning directly in torque space can

induce highly non-smooth exploration when contact constraints and deformable dynamics are present [?].

Action spaces compatible with IsaacLab terms. IsaacLab explicitly supports action terms that map normalized policy outputs to robot commands via an action manager [?, ?]. This interface enables several action parameterizations relevant to cloth grasp-and-stretch:

- **Joint-level continuous control:** Actions are mapped to joint targets (e.g., position or velocity) for selected joint subsets through configured action terms [?]. The general batched action tensor is processed per environment instance and applied during the physics step loop [?].
- **Cartesian end-effector motion primitives:** Actions are defined as incremental end-effector translations/rotations (and gripper commands), which are converted to joint targets through inverse kinematics or task-space controllers available in IsaacLab/Isaac Sim toolchains [?, ?]. This abstraction aligns with common cloth manipulation formulations that predict pick points and pull vectors rather than raw joint torques [?].
- **Robust gripper open/close actions:** For grasping-heavy tasks, IsaacLab provides an action term that converts a continuous gripper input into binary open/close commands using a threshold mechanism, with the stated aim of producing more predictable grasping behavior [?]. This mapping is suitable when grasp success is more sensitive to discrete state (open vs. closed) than to continuous gripper width modulation.

For cloth manipulation with a parallel-jaw gripper, a practical action vector is often defined as

$$\mathbf{a}_t = [\Delta \mathbf{x}_t^\top \quad \Delta \boldsymbol{\phi}_t^\top \quad a_t^{\text{grip}}]^\top,$$

where $\Delta \mathbf{x}_t \in \mathbb{R}^3$ and $\Delta \boldsymbol{\phi}_t \in \mathbb{R}^3$ denote incremental Cartesian translation and rotation commands, and a_t^{grip} denotes a gripper command (continuous or binary) [?, ?]. In IsaacLab, such a design is realized by composing multiple action terms under the action manager, allowing joint-space actuation and gripper commands to be processed consistently in a single step [?, ?].

2.5.3.3 Reward function design

Success criteria for cloth grasping and stretching. Reward design for cloth manipulation is difficult because task success is geometric and often sparse (e.g., “the cloth is sufficiently spread”), while exploration under contact and occlusion is challenging [?, ?]. In deformable-object benchmarks, spreading or smoothing tasks have been operationalized using coverage/area metrics computed from top-down projections of the cloth configuration [?]. Similarly, sequential fabric smoothing work has used the objective of maximizing cloth coverage as a success measure when learning pick-and-pull actions from visual observations [?].

Dense reward decomposition. To support stable learning, a dense reward can be constructed as a weighted sum of interpretable terms:

$$r_t = w_{\text{grasp}} r_t^{\text{grasp}} + w_{\text{stretch}} r_t^{\text{stretch}} + w_{\text{align}} r_t^{\text{align}} - w_{\text{act}} r_t^{\text{act}} - w_{\text{viol}} r_t^{\text{viol}}, \quad (2.11)$$

where each term targets a specific behavioral component and weights are tuned to balance shaping strength [?, ?].

A concrete instantiation suitable for grasp-and-stretch includes:

- **Grasp acquisition** r_t^{grasp} : A sparse positive reward is assigned when the cloth is successfully grasped (e.g., when the gripper is closed and cloth contact/grasp conditions are satisfied), since grasp failure prevents downstream stretching behavior. This mirrors the emphasis on successful pick selection in cloth shaping pipelines [?].
- **Spreading/stretching** r_t^{stretch} : A natural choice is a coverage-based metric, such as the top-down area covered by the cloth, which is explicitly used as the reward in the SoftGym `SpreadCloth` task [?]. When the task is formulated as stretching between grasp points (e.g., two corners), r_t^{stretch} may alternatively be defined as the increase in distance between target cloth keypoints, consistent with pick-and-pull vector formulations [?].
- **Alignment/placement** r_t^{align} : If the cloth is required to be stretched into a target pose (e.g., aligned on a conveyor or within a region of interest), an auxiliary term can penalize deviations between current and desired keypoint positions or between the cloth centroid and a target centroid. Distance-based terms are widely used in deformable manipulation benchmarks that compare current particle positions to a target configuration [?].
- **Action regularization** r_t^{act} : Smoothness penalties (e.g., $\|a_t\|^2$) or effort penalties discourage high-frequency control signals and can improve numerical stability and

hardware transfer compatibility [?].

- **Constraint/safety violations** r_t^{viol} : Penalties can be applied when joint limits are approached, self-collisions occur, or the cloth leaves the workspace, complementing termination logic and discouraging destructive exploration [?].

In IsaacLab manager-based tasks, each component in Equation 2.11 is implemented as a reward term with an associated weight and optional parameters, enabling systematic ablations and incremental shaping [?, ?]. This design is particularly aligned with deformable tasks where a single sparse success reward is often insufficient to drive exploration [?, ?].

2.5.3.4 Episode termination conditions

Timeout and truncation. Finite-horizon episodes are typically enforced through a maximum episode length T to bound return estimation and stabilize training statistics [?]. IsaacLab provides a standard termination term that triggers episode termination when the episode length exceeds the configured limit (`time_out`) [?]. In vectorized execution, environment stepping is designed such that terminated sub-environments are reset internally by the `step()` implementation, and it is therefore recommended that `reset()` is called only once prior to the first `step()` call [?].

Success termination. A success condition for cloth grasp-and-stretch is most naturally defined as achieving a threshold on a geometric metric, such as: (i) area coverage exceeding a fraction of the maximum possible area (coverage-based completion) [?], or (ii) keypoint alignment error falling below a tolerance (target-configuration completion) [?]. In a manager-based implementation, such conditions are introduced as termination terms (custom or built-in) evaluated each step by the termination manager [?, ?].

Failure termination. To prevent unproductive exploration and to approximate physical safety constraints, failure termination conditions are typically included, such as:

- **Robot constraint violations:** Episode termination when joint positions exceed soft limits is supported by built-in termination primitives (e.g., `joint_pos_out_of_limit`) [?].
- **Task failure events:** Termination when the cloth leaves the workspace, when grasp is lost irrecoverably (e.g., cloth is dropped and not re-grasped within a horizon), or when excessive collisions occur. These conditions are task-specific and are typically implemented as custom termination terms [?].

In addition to providing episode boundaries, termination events can be incorporated into the reward via explicit termination penalties, which are supported as reusable reward terms that penalize non-timeout terminations [?].

2.5.4 Reward Shaping and Curriculum Strategies

Reward shaping and policy invariance. Reward shaping refers to modifying or augmenting the environment reward signal to accelerate learning by providing denser feedback for intermediate behaviors [?]. In Reinforcement Learning theory, a central concern is whether reward transformations preserve the optimal policy. A sufficient condition is potential-based shaping, in which an additional shaping reward is constructed from a potential function $\Phi : \mathcal{S} \rightarrow \mathbb{R}$ such that

$$F(s_t, s_{t+1}) = \gamma \Phi(s_{t+1}) - \Phi(s_t), \quad (2.12)$$

and the shaped reward is defined as $r'_t = r_t + F(s_t, s_{t+1})$ [?, ?]. It has been shown that, under this construction, the optimal policy of the shaped Markov Decision Process is preserved while learning dynamics and exploration behavior can be improved [?]. This principle is particularly relevant for cloth tasks, where sparse terminal rewards (e.g., “cloth fully stretched”) provide limited guidance during exploration in high-dimensional state spaces [?, ?].

Curriculum learning. Curriculum learning formalizes the idea that training can be accelerated by presenting tasks in an order of increasing difficulty, rather than sampling uniformly from the full difficulty distribution from the outset [?]. For deformable-object manipulation, curriculum strategies are frequently motivated by the large configuration space and the dependence of success on first achieving prerequisite behaviors (e.g., reliable grasp before stretching) [?]. In a cloth grasp-and-stretch setting, a curriculum can be instantiated by gradually increasing: (i) initial cloth disorder (wrinkles, random folds), (ii) friction/contact difficulty, (iii) domain randomization ranges (material parameters, observation noise), or (iv) task precision requirements (tighter alignment tolerances).

Curriculum support in IsaacLab. IsaacLab natively supports curriculum specification through a curriculum manager, and includes built-in curriculum terms that modify reward weights or environment parameters according to step-wise schedules (e.g., `modify_reward_weight`, `modify_env_param`) [?]. This enables curriculum learning to be operationalized without restructuring the core environment logic: shaped reward weights

can be increased as competence improves, and environment parameters controlling task difficulty can be updated online during training [?, ?]. In the manager-based workflow, such curricula are configured alongside observation, action, reward, and termination specifications, supporting controlled experimentation with staged training protocols [?, ?].

2.5.5 GPU-Vectorized Training and Environment Wrappers

Vectorized environments on the GPU. High sample throughput is essential for deep Reinforcement Learning in robotics, particularly for contact-rich tasks with deformable dynamics. GPU-accelerated simulation frameworks have demonstrated substantial speedups by executing many environment instances in parallel and by avoiding CPU-GPU transfer bottlenecks between simulation and learning [?, ?]. Isaac Gym established an end-to-end GPU pipeline in which physics simulation and neural network training reside on the GPU and communicate through tensor interfaces, enabling large-scale batched roll-outs [?]. IsaacLab is positioned as the successor to this paradigm, extending GPU-native simulation with additional capabilities (e.g., richer sensors, actuator models, and support for deformable objects) while remaining oriented toward large-scale parallel learning [?].

In IsaacLab, Reinforcement Learning environments are designed to run as vectorized simulations with a configurable number of sub-environments (`num_envs`), where observations and actions are batched per step [?]. The environment implements a Gym-like interface, but it is not itself a `gym.vector.VectorEnv`; instead, wrappers are used to adapt to the varying vector-environment conventions of different learning libraries [?, ?].

Environment wrappers and library integration. The OpenAI Gym interface popularized the concept of wrapping environments to modify observations, rewards, and logging behavior without changing the underlying task implementation [?]. IsaacLab follows this design pattern and states that Reinforcement Learning environments inheriting from its base classes implement the `gymnasium.Env` interface and are compatible with `gymnasium.Wrapper` [?]. Wrappers are used both for generic purposes (e.g., enforcing call order, recording videos) and for adapting to Reinforcement Learning library-specific APIs [?, ?].

IsaacLab explicitly provides wrappers that convert the environment I/O to match the expectations of multiple Reinforcement Learning libraries (e.g., vectorized environment APIs that require particular tensor/array formats) and recommends applying learning-framework wrappers last, since such wrappers may modify interpretation of environment calls [?, ?]. This design preserves a single environment implementation while enabling

interchangeability of training backends, which is consistent with the manager-based objective of modularity and reuse [?, ?].

Scene replication for parallelism. Vectorization at the simulation level is enabled not only by batched stepping but also by scene replication. Isaac Sim provides a “Cloner” interface designed to simplify constructing scenes with many replicated environment instances and includes utilities for generating clone paths, computing transforms, and filtering collisions between clones [?]. The same documentation emphasizes that such vectorized copies are beneficial for collecting trajectories in parallel during Reinforcement Learning training [?]. Together with IsaacLab’s batched environment APIs, this supports scalable data collection for policy learning in simulation [?, ?].

2.6 Prior Work

2.7 Robot and Scene Description Formats

Robotic learning workflows in simulation typically require at least two layers of model description: (i) a *robot model* capturing kinematic structure and physical properties, and (ii) a *scene/world model* capturing the placement of robots and objects, lighting and sensors, and simulator-specific physics configuration. In the Universal Scene Description-based *Omniverse* ecosystem, the scene is represented as a composed Universal Scene Description stage, while robots may be authored in robotics-oriented formats such as Unified Robot Description Format or MuJoCo XML Format and subsequently imported into Universal Scene Description for simulation. This section explains the role of Universal Scene Description, Unified Robot Description Format, and MuJoCo XML Format in this separation of concerns and clarifies how these formats are used in this thesis.

Placeholder for *format conversion and integration pipeline* image

Figure 2.6: Schematic pipeline illustrating how robot descriptions (e.g., Unified Robot Description Format or MuJoCo XML Format) are converted into Universal Scene Description assets and integrated with environment elements (rigid objects, cloth, lights, sensors) on a composed Universal Scene Description stage.

2.7.1 USD as a scene graph for simulation worlds

Core scene description concept. Universal Scene Description is a system for encoding scalable, hierarchically organized data (static as well as time-sampled) with the primary purpose of interchange and augmentation across digital content creation applications [?]. Beyond being a file encoding, Universal Scene Description is designed to *compose* a single stage from scene description distributed across many layers and assets, supporting sparse overrides and asset aggregation via composition operators such as references and variants [?]. This composition-centric design is the main reason Universal Scene Description is used as the foundation of *NVIDIA Omniverse*-based applications, including robotics simulation: a simulation world can be assembled from reusable assets (robots, fixtures, conveyors, cloth objects), while experiment-specific changes (randomization parameters, object placements, material settings) can be authored as non-destructive overrides.

Extensibility via schemas. A central mechanism in Universal Scene Description is *schema-based extensibility*: standardized or custom schemas define what properties are available on a prim and how data are interpreted by downstream tools [?, ?]. For robotics simulation, this is crucial because a scene must not only store geometry and transforms, but also physics and sensor semantics. In practice, simulation-ready Universal Scene Description stages rely on multiple cooperating schema families: (i) rendering-related schemas (e.g., light prims), and (ii) physics-related schemas that define rigid bodies, collisions, articulation joints, and constraints, enabling a simulator backend to interpret the stage as a dynamical system.

Lighting and sensor-relevant elements. Lighting in Universal Scene Description is represented by the Universal Scene Description lighting schema *UsdLux*, which provides a portable representation for common light types and attributes (e.g., intensity/exposure and color) intended for interchange across rendering environments [?]. This is directly relevant for robotics simulation, because perception pipelines and synthetic data generation depend on reproducible lighting setups. The use of common light schemas also reduces simulator lock-in at the scene-description level, since the same lighting structure can be understood by multiple Omniverse tools and render backends [?].

Physics properties encoded in USD. Physics semantics in Universal Scene Description are provided by *UsdPhysics*, which defines prim and property schemas for representing physics simulation state and configuration on a stage [?]. Within this schema

family, rigid bodies, joints, and constraints are described as Universal Scene Description prims and applied Application Programming Interface schemas, allowing physics engines to identify which parts of the stage are simulated and how they are constrained [?]. Joint actuation is commonly represented via drive attributes that behave as implicit damped springs, i.e., forces/accelerations are defined via stiffness and damping terms toward a target position/velocity [?]. In Omniverse, additional simulator-specific tuning parameters are exposed through *PhysX schema additions*; for example, PhysX joint schemas provide per-axis parameters and actuator performance characteristics extending the generic *UsdPhysics* joint representation [?].

2.7.2 URDF as a robotics-oriented robot model format

Scope and typical usage. Unified Robot Description Format is an Extensible Markup Language-based format used in the *Robot Operating System* ecosystem for specifying robot geometry and organization [?]. A Unified Robot Description Format model typically focuses on the robot as a kinematic structure, described by rigid links connected through joints, plus the geometric information used for visualization and collision checking, and the physical properties required for rigid-body simulation (e.g., link inertias) [?]. This focus makes Unified Robot Description Format a common interchange format for robotic manipulators and mobile robots, especially when robots are also integrated into *ROS* pipelines for control, state publishing, and visualization [?].

Kinematic structure, joint limits, and inertial properties. Within the Unified Robot Description Format workflow, links are augmented with collision geometry and inertial parameters, and joints are augmented with joint dynamics and limits. The ROS 2 Unified Robot Description Format tutorials explicitly emphasize that collision and inertial properties are needed for simulation, and that joint dynamics parameters (e.g., friction) can be specified as part of the joint description [?]. Joint limits (position bounds, effort/velocity bounds) are also represented at the joint level in Unified Robot Description Format-based tooling, and are typically used both for controller safety and for ensuring physically plausible simulation states [?].

Structural limitations relevant to tendon mechanisms. A key limitation of Unified Robot Description Format is that, in standard robotics tooling, only serial kinematic chains are supported in a direct and portable way; more complex kinematic dependencies are generally handled through additional simulator- or controller-specific mechanisms [?].

A commonly supported exception is the *mimic joint* concept, which allows joint coordinates to be coupled through a linear dependency. Mimic joints are explicitly described as a pragmatic abstraction used to represent simplified closed-loop dependencies or mechanically coupled degrees of freedom, such as belt-and-pulley systems [?]. This capability is relevant for tendon-driven or cable-driven mechanisms when coupling can be approximated as a simple linear relationship. However, tendon routing with wrapping, varying moment arms, elastic cable stretch, frictional loss, and multi-joint coupling is generally not fully representable with the standard Unified Robot Description Format kinematic abstraction alone [?]. In such cases, either simulator-specific extensions or higher-level actuation models must be introduced.

2.7.3 MJCF as MuJoCo’s native modeling language

Positioning and expressiveness. MuJoCo XML Format is the native Extensible Markup Language modeling language of *MuJoCo* and is designed to expose a rich set of model elements while remaining scalable through default mechanisms and compile-time processing [?]. In the original MuJoCo description, model construction is performed either through a high-level Extensible Markup Language model file (MuJoCo XML Format) or programmatic Application Programming Interface calls, with the MuJoCo XML Format file intended as an intuitive text description of the model to be compiled into the simulator’s internal representation [?]. Importantly, MuJoCo documentation explicitly describes Unified Robot Description Format as “popular but more limited”, and positions MuJoCo XML Format as the modeling format used to access MuJoCo’s full expressiveness [?].

Tendons as first-class elements. For tendon-driven robots, a central feature of MuJoCo’s modeling language is that tendons are directly supported as model elements. MuJoCo documentation describes tendon geometry as minimum-path-length strings with wrapping and via-point constraints, and notes that robotics-specific structures such as pulleys and coupled degrees of freedom are provided as part of the tendon mechanism [?]. Tendons can be used for actuation as well as to impose inequality/equality constraints on tendon length [?]. This contrasts with standard Unified Robot Description Format usage, where cable routing and tendon geometry are not part of the core, portable robot description abstraction [?, ?].

Relationship to URDF-based models. MuJoCo tooling supports loading both MuJoCo XML Format and Unified Robot Description Format descriptions, while explicitly stating that the Unified Robot Description Format representation covers only a subset

of the available model elements and features [?]. Consequently, MuJoCo XML Format is often selected when the simulation must include advanced actuation and constraint structures (including tendons) that are difficult to express in Unified Robot Description Format without simulator-specific extensions [?, ?].

2.7.4 Format use in this thesis and tendon-driven modeling workarounds

Responsibilities of each format in the pipeline. In the context of Isaac Sim and Isaac Lab, Universal Scene Description is treated as the *runtime scene format*: robots and environment assets are ultimately represented as Universal Scene Description so that they can be loaded, referenced, and simulated on a Universal Scene Description stage [?]. Robot-centric description formats such as Unified Robot Description Format or MuJoCo XML Format are primarily used as *authoring or interchange formats* for the robot model itself. Their primary responsibility is to capture: (i) robot topology (links and joints), (ii) joint types and joint limits, and (iii) rigid-body physical properties (e.g., inertial parameters and collision geometry) required for simulation [?, ?]. By contrast, Universal Scene Description is used to integrate the robot with the rest of the simulated world and to attach simulation-specific semantics (physics schema applications, light prims, and renderer-facing scene elements) [?, ?, ?].

Table 2.5: Roles of Universal Scene Description, Unified Robot Description Format, and MuJoCo XML Format in robotics simulation pipelines.

Format	Primary scope
Universal Scene Description	Scene graph with composition
Unified Robot Description Format	Robot kinematics + basic rigid-body data
MuJoCo XML Format	Full MuJoCo model description
	Expressive robot model

Which format is used for the tendon-driven robot and why. In this thesis, the custom tendon-driven arm component is provided as a *modified* Unified Robot Description Format description and is imported into Isaac Sim, where it becomes a Universal Scene Description robot asset for simulation. This choice is motivated by the practical availability of Unified Robot Description Format as a widely used robotics interchange format and by the fact that Isaac Sim provides an explicit import workflow that converts Unified Robot Description Format robot descriptions into Universal Scene Description

assets for use on a Universal Scene Description stage [?]. In addition, Isaac Lab recommends converting external robot model formats into Universal Scene Description using the provided importers (including Unified Robot Description Format and MuJoCo XML Format importers) and then performing subsequent editing, tuning, and large-scale simulation using the Universal Scene Description representation [?]. The final integrated robot is therefore simulated as Universal Scene Description within Isaac Sim, consistent with Omniverse’s Universal Scene Description-native scene representation [?].

Integration of robot and scene as USD assets. Once the robot components exist as Universal Scene Description assets, integration into a complete simulation scene is performed by placing them on the Universal Scene Description stage together with the environment geometry and task objects (including cloth and supporting structures) and then attaching physics and rendering semantics through the relevant schemas [?, ?]. In Omniverse-based robotics workflows, modular robot assemblies are commonly created by combining separate Universal Scene Description robot assets (e.g., a base, an arm, and a gripper) via fixed joints. Isaac Sim’s *Robot Assembler* tool is explicitly designed for this use case: it combines two Universal Scene Description assets through a physically simulated fixed joint and produces a resulting Universal Scene Description asset that can be saved and loaded as a single composite robot [?].

URDF modifications and tendon modeling workarounds. Standard Unified Robot Description Format does not provide a portable, first-class representation of tendon routing with wrapping, elastic elements, and multi-joint cable coupling comparable to MuJoCo XML Format’s tendon mechanism [?, ?]. Therefore, tendon-driven mechanisms are often approximated in Unified Robot Description Format-to-simulator pipelines by mapping tendon actuation to joint-level drive targets (position/velocity/effort) and by encoding only those couplings that can be captured with simple kinematic relationships (e.g., mimic joints) [?]. In Omniverse workflows, practical Unified Robot Description Format-to-Universal Scene Description conversion constraints further motivate simplification: Isaac Lab explicitly documents that some Unified Robot Description Format tags used in other simulators (e.g., `<transmission>` and `<gazebo>` tags) are not supported by the Unified Robot Description Format importer, and that pre-processing may therefore remove such tags prior to conversion [?]. This limitation is particularly relevant for tendon-driven robots if tendon routing or actuation coupling has been encoded via transmission-like blocks: such elements may need to be removed or replaced by simulator-native constructs after import.

In the present thesis draft, the tendon routing/transmission constructs in the imported

arm description are simplified such that each tendon-driven joint is represented as an actuated revolute joint driven directly by joint targets rather than an explicit cable model. Moreover, joint limits are enforced in the description to improve stability, while complex tendon dynamics (stretch, multi-joint coupling, backlash) are not explicitly simulated at this stage.

Why PhysX tendon features are not used in the current implementation. Although PhysX articulations support tendon constraints at the articulation level, including fixed tendons (constraints on joint positions) and spatial tendons (distance constraints along attachment points) [?], limitations in simulation fidelity and behavior have been documented for articulation tendons in Omniverse Physics. Specifically, articulation tendons are listed among known limitations, and workarounds are recommended: mimic-joint style coupling is suggested for fixed tendon behavior, and external forces are suggested to mimic spatial tendon effects [?]. In this thesis, these limitations motivate a simplified approach in which tendon effects are approximated via direct joint drives (spring-damper-like actuation toward joint targets) rather than via explicit PhysX tendon constraints, thereby prioritizing stability and controllability of the learning environment over detailed tendon mechanics.

- **URDF role:** The tendon-driven arm geometry, kinematic chain, joint limits, collision shapes, and rigid-body inertial properties are authored in Unified Robot Description Format and then converted for Omniverse simulation [?, ?].
- **USD role:** The robot (after conversion) and the environment assets are integrated on a composed Universal Scene Description stage; physics and lighting semantics are expressed via *UsdPhysics* and *UsdLux*, and PhysX-specific parameters are expressed via PhysX schema additions [?, ?, ?].
- **MJCF role:** MuJoCo XML Format is treated as a common alternative robot description format in the broader robotics and learning literature, with particularly strong support for tendon mechanisms in MuJoCo; it is not used as the primary authoring format in this thesis, but conversion into Universal Scene Description is supported in Isaac Lab when MuJoCo XML Format assets exist [?, ?, ?].
- **Tendon workarounds:** Due to both representational limitations of Unified Robot Description Format and documented limitations of articulation tendon fidelity in Omniverse Physics, tendon actuation is approximated through joint drives and coupling simplifications rather than explicit tendon constraints [?, ?].

3 Research Gap

3.1 Critical Analysis of State of the Art

3.2 Research Gap and Justification

3.3 Refined Problem Definition

3.4 Success Criteria

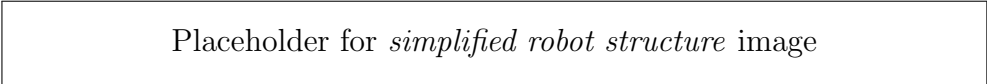
4 Methodology

4.1 Overview of Approach

4.2 Simulation Environment Setup

4.3 Robot Model Integration

In this section, the simulation setup for the custom tendon-driven robotic manipulator is presented. The complete robot model (Figure ??) consists of three modular components: a 2-Degree of Freedom articulated base providing linear motion, a 3-Degree of Freedom tendon-driven arm mounted atop the base, and a Robotiq 2F-140 two-finger gripper as the end-effector. All components are integrated in NVIDIA Isaac Sim 5.1.0, under the project’s Methodology and Implementation framework, to emulate the physical robot’s behavior in a virtual environment. The following subsections detail each component’s design and integration, the assembly of the full manipulator using Isaac Sim’s tools, and the validation of its control setup.



Placeholder for *simplified robot structure* image

Figure 4.1: Simplified structure of the integrated robot model in Isaac Sim, showing the 2-DoF linear base, the 3-DoF tendon-driven arm, and the Robotiq 2F-140 gripper attached as end-effector.

4.3.1 2-DoF Articulated Base (Linear Y–Z Stage)

The articulated base is a two-degree-of-freedom linear stage that provides planar positioning for the arm along the horizontal Y -axis and vertical Z -axis. It is designed exclusively in simulation, with no direct real-world counterpart, serving as a versatile foundation to adjust the arm’s position within the workspace. The base’s prismatic joints allow translation of the arm $\pm 0.4\text{m}$ laterally (Y direction) and -0.5m downward in height (Z direction) from a defined origin (with $Z = 0$ at the upper limit) - see Table?? for a summary of joint travel ranges. These limits yield a horizontal span of 0.8m and a vertical adjustment of

0.5 m, chosen to cover the required operating area of the project’s frame while avoiding singular configurations or collisions at extremes.

In implementation, the base was constructed from scratch within Isaac Sim using the native Universal Scene Description robot schema definitions (rather than importing from Unified Robot Description Format). It consists of rigid link geometry (modeled as simple shapes) and two linear actuators defined as prismatic joints along the global Y and Z axes. Each prismatic joint is configured with a physics drive (stiffness and damping) for position control, and appropriate limit stops at the specified travel range. The entire base assembly was built to fit the physical dimensions of the project’s frame and mounting interface, ensuring that when the arm is attached, its workspace aligns with the intended real-world environment. Because the base is purely virtual, it enabled rapid prototyping of different mounting heights and lateral positions for the arm without hardware modifications. All structural components of the base were kept simple and lightweight in simulation to maintain real-time performance.

4.3.2 3-DoF Tendon-Driven Arm

Mounted on the base’s vertical slider is the tendon-driven manipulator arm with three actuated degrees of freedom. This arm’s design and geometry are derived from a prior Computer-Aided Design model developed in a master’s thesis by Klein (2023)[?]. The original arm is a cable-driven (tendon-driven) mechanism inspired by a human arm’s musculo-tendon structure, featuring an upper arm, forearm, and wrist. In our simulation, the arm was imported via a modified Unified Robot Description Format file based on Klein’s Computer-Aided Design model. The Unified Robot Description Format was adjusted for compatibility with IsaacSim - for example, custom transmission elements representing the tendon routing were simplified or removed, and each tendon-driven joint was defined as a standard revolute joint with direct actuation. The arm is rigidly attached to the top of the base (on the moving Z -axis platform) so that its shoulder reference frame coincides with the base’s moving frame.

Each of the arm’s three joints is modeled in IsaacSim as a revolute articulation with a position/force drive to emulate the tendon actuation. In the physical design, these joints would be driven by tendons via remote actuators, but in simulation they are controlled directly by setting joint angle targets and applying spring-damper forces. The first joint (shoulder/elbow pitch) allows approximately 172° of rotation ($\pm 86^\circ$, i.e. ± 1.5 rad, from neutral), corresponding to the primary bending of the “elbow” in the vertical plane as described by Klein [?]. The wrist mechanism of the arm is implemented as two consecutive revolute joints (perpendicular axes) representing the wrist’s flexion/extension and

abduction/adduction; each of these has a range of $\pm 46^\circ$ (± 0.8 rad). These ranges are in line with the mechanical constraints reported in the original design [?]. For simulation stability, hard joint limits are imposed in the Unified Robot Description Format/Universal Scene Description definition to prevent the solver from pushing the joints beyond their intended travel.

Tendon Actuation Modeling: Instead of using PhysX’s built-in tendon simulation feature for articulations, the tendon-driven joints are simulated with direct joint drives. NVIDIA’s documentation notes that the PhysX articulation tendon feature can exhibit inadequate fidelity and unstable behavior under certain conditions [?]. In particular, spatial tendon constraints may lead to solver instabilities or unrealistic force feedback. To avoid these issues, we opted not to use PhysX “physxTendon” elements. Instead, each joint is given a Proportional-Derivative control (a stiff spring with damping) that attempts to maintain the target angle, thereby mimicking the effect of a pretensioned tendon pulling the joint into position. The drive stiffness and damping values were initially chosen based on the estimated elasticity and friction in the tendons, and later fine-tuned using the Gain Tuning tool (see Section ??). This approach provides a stable yet simplified representation of tendon actuation: the joints behave as if driven by motors with compliance, without modeling cable stretch or complex tendon dynamics. We acknowledge that the tendon dynamics (e.g., stretch, backlash, and multi-joint coupling) are not explicitly simulated at this stage - incorporating a more realistic tendon model or a dedicated cable simulation is left for future work.

Collision Geometry: The arm’s collision shapes were generated using convex hull approximations of the Computer-Aided Design model geometry. Each link (upper arm, forearm, wrist links) is assigned a convex collision mesh computed from its visual mesh. Using convex hulls simplifies the collision geometry and is recommended for better physics performance in Isaac Sim. This ensures robust contact simulation while avoiding the computational cost and potential instability of complex mesh collisions. The convex colliders reasonably approximate the arm’s shape (covering joint protrusions, etc.) and were verified to avoid interpenetration in all valid joint configurations. With these collision models in place, the arm can interact with the environment (e.g., grasped objects or potential obstacles) in simulation without excessive physics solver load.

4.3.3 Robotiq 2F-140 Gripper Integration

For the end-effector, the Robotiq 2F-140 adaptive gripper was selected from IsaacSim’s built-in asset library. This is a two-finger parallel gripper well-suited to the project’s manipulation tasks. The 2F-140 model was chosen for its wide stroke and proven com-

patibility: it can grip a range of object sizes with an adjustable finger span up to 140mm, and it provides a significant grip force for secure holding. According to the manufacturer’s specifications, the 2F-140 offers a maximum stroke of 140mm and an adjustable grip force from about 10N up to 125N[?]. It is rated for handling payloads up to roughly 2.5kg in a form-fit grasp, while the gripper’s own weight is about 1kg. These characteristics make it capable of picking and holding typical objects in our application (such as mixed waste items) without exceeding the arm’s load capacity.

In simulation, the Robotiq gripper is loaded as a self-contained articulated asset (with its finger linkage and drive mechanism pre-defined by NVIDIA’s asset). This greatly simplified integration: rather than modeling a custom gripper, the ready-made 2F-140 module was attached to the robot arm and controlled through its single actuated degree of freedom (the synchronized motion of the two fingers). The gripper’s coupling interface follows the standard ISO-50 flange, and it was directly compatible with the mounting point on the wrist of the tendon-driven arm. Notably, this gripper has been used in other Isaac Sim and Isaac Lab setups (e.g., with Universal Robots UR10 arms), indicating robust support and reliability in the simulation environment. Its inclusion here not only meets the functional requirements (adaptive gripping of various objects) but also leverages existing calibration and tuning knowledge from prior implementations. The official product sheet and instruction manual provided further technical details and usage guidelines, which were consulted to ensure the simulated gripper’s behavior (speed, force, fingertip friction, etc.) aligns with the real device [?].

4.3.4 Modular Assembly via Isaac Sim’s Robot Assembler

With the base, arm, and gripper defined as separate components, the complete robot was assembled within IsaacSim using the Robot Assembler tool. The Robot Assembler is an interactive utility provided by IsaacSim that allows users to combine multiple robot sub-assemblies into a single coherent model by creating fixed joints between them. In our case, we performed a two-step assembly: (1) attaching the 3-Degree of Freedom tendon-driven arm onto the linear base, and (2) attaching the 2F-140 gripper onto the arm’s wrist flange. IsaacSim’s documentation explicitly supports these use cases - for example, assembling a robot arm to a moving base, or attaching a gripper to a robot arm, are typical workflows[?]. Using the Robot Assembler UI (accessible via Tools → Robotics → Asset Editor → Robot Assembler in Isaac Sim), the base was designated as the “Base Robot,” and the arm was added as the “Attach Robot” at the appropriate attach point on the base (the top plate of the Z -axis slider). The attach point corresponded to a pre-defined locator frame on the base where the arm’s bottom joint should coincide. Once the arm was

assembled to the base, the combined base+arm was treated as a new single robot. Then, similarly, the gripper asset was attached to the arm: the arm’s Unified Robot Description Format included a tool flange link at the wrist, which we used as the attach point for the gripper (with the gripper as “Attach Robot” in the tool). The assembler automatically created a fixed joint linking the gripper’s base to the arm’s flange, aligning them based on their coordinate frames. After each assembly operation, the resulting composite robot (base+arm+gripper) was saved to a new Universal Scene Description file so that it can be loaded in the future as one complete manipulator model without needing to repeat the manual assembly steps.

Because the Robot Assembler uses a physically simulated fixed joint to bond the parts, the attached components behave as a single articulated body during simulation. This means the base, arm, and gripper move together without drift when physics is enabled, but they also remain modular in the project structure. One major advantage of this modular assembly approach is the flexibility for future hardware replacement or reconfiguration. Each subsystem (base, arm, gripper) can be swapped out or modified independently. For instance, if a different gripper model is to be tested in the future, it can replace the 2F-140 in the assembly with minimal effort; or if the project evolves to use a physical linear axis or a different arm, those could be integrated by the same method. The assembled manipulator’s design is therefore not monolithic - it is reconfigurable by design. Figure ?? illustrates an exploded view of the robot, showing the three modules (linear base, tendon arm, and gripper) and their attachment relationships. Such a view highlights the modular interfaces: the mounting plate between base and arm, and the wrist flange between arm and gripper.

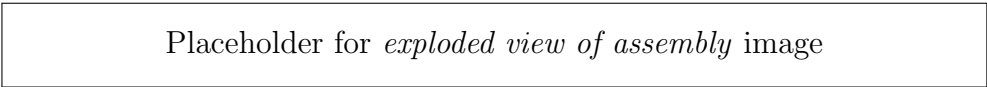


Figure 4.2: Exploded view of the robot assembly, showing (from bottom) the 2-DoF linear base, the 3-DoF tendon-driven arm, and the Robotiq 2F-140 gripper. The arrows indicate the attachment points: the arm mounts onto the base’s top slide, and the gripper attaches to the arm’s wrist flange. This modular design allows each component to be independently developed or replaced.

After assembly, the final integrated robot has a total of five controlled joints (two prismatic joints in the base and three revolute joints in the arm, not counting the underactuated motion of the gripper’s fingers which are driven by one actuator). Table ?? summarizes the joint types and their ranges of motion in the integrated model. These ranges reflect both the mechanical design limits and the safety constraints imposed in simulation. By verifying the combined kinematic chain, we ensured that the robot can reach the required workspace for the intended tasks (e.g., covering the sorting area and adjusting to various

bin heights) while respecting joint limits. All joints are configured with appropriate drive parameters and limit enforcement as discussed previously. In the next subsection, we address how the control gains for these joints were tuned and validated.

Table 4.1: Actuated joints of the integrated robot and their motion ranges.

Joint	Type & Axis	Range of Motion
Base Linear Y	Prismatic (horizontal slide)	−0.4 m to +0.4 m
Base Linear Z	Prismatic (vertical lift)	−0.5 m to 0.0 m (downwards)
Arm Joint 1	Revolute (shoulder/elbow pitch)	−86° to +86° (± 1.5 rad)
Arm Joint 2	Revolute (wrist tilt X-axis)	−46° to +46° (± 0.8 rad)
Arm Joint 3	Revolute (wrist tilt Y-axis)	−46° to +46° (± 0.8 rad)

4.3.5 Control Gain Tuning and Validation

Once the robot model was assembled, a critical step was to validate and tune the joint control gains to achieve stable and accurate motion. We utilized IsaacSim’s Gain Tuner extension for this purpose. The Gain Tuner provides tools to adjust the proportional position stiffness and damping parameters of each joint’s drive and to test the robot’s response to control inputs in simulation. According to NVIDIA’s documentation, the Gain Tuner is intended to help find a pair of stiffness and damping gains for each joint such that the robot can follow commanded trajectories with the expected behavior[?]. In practice, we performed automated gain testing on all five joints: the tool applied small-amplitude sinusoidal position commands and step changes within each joint’s allowable range, while we observed the joint’s motion response. For each joint, the stiffness (analogous to a P-gain) was increased until the joint responded quickly to commands without significant steady-state error, and the damping (D-gain) was adjusted to critically damp the response (minimizing overshoot and oscillation). Isaac Sim’s Gain Tuner provided visual feedback in the form of plots comparing commanded vs. actual joint trajectories, which was invaluable for iterative tuning and verifying stability.

Through this gain tuning process, we confirmed that the base’s linear axes and the arm’s revolute joints could be moved under position control with smooth and stable behavior. In particular, the heavy vertical axis (base Z) required a higher damping ratio to counteract gravity-induced oscillations, and the relatively lightweight wrist joints could use lower stiffness to reflect their tendon-driven compliance. After tuning, the robot’s joints exhibit near-critical damping: when given a step command, each joint reaches the target quickly

(within 0.1-0.3 s settling time) with minimal overshoot (typically $< 5\%$ of the motion range) and no sustained oscillation. These tuned gains were then fixed in the simulation configuration for all subsequent experiments. This control validation step was essential to ensure that high-level control or planning algorithms (addressed in later chapters) would interact with a robot model that behaves in a realistic and predictable manner. By using the Gain Tuner, we effectively calibrated the simulated actuators to mimic a well-tuned physical controller, thereby laying a sound foundation for the robot’s performance in simulation.

4.4 Reinforcement Learning Setup

4.4.1 State Space

4.4.2 Action Space

4.4.3 Reward Space

4.4.4 RL Algorithm and Training Parameters

4.4.5 MDP Workflow in IsaacLab

4.5 Initial Experiments

To work towards the final implementation, initial experiments were conducted focusing on basic tasks such as reaching a specified pose and performing simple pick-and-place operations. These experiments served to validate the robot model and the reinforcement learning setup within the simulation environment. As comparison points, baseline results example implementations from IsaacLab with the UR10 robot arm were utilized.

4.5.1 Reach Pose Task

The reach pose task involved training the tendon-driven robot to move its end-effector to a designated position in 3D space. The performance was evaluated based on the accuracy of reaching the target position within a defined tolerance.

4.5.2 Simple Pick and Place Task

In the simple pick-and-place task, the robot was trained to grasp an object from one location and place it at another specified location. The success of this task was measured by the robot's ability to securely grasp the object and accurately place it at the target location.

4.5.3

4.6 Advanced Task Implementation

4.7 Software and Hardware Integration

5 Results

5.1 Preliminary Experiments

5.2 Performance of Tendon-Driven Robot on Simple Tasks

5.3 Cloth Manipulation Task Results

5.4 Summary of Key Results

6 Discussion

6.1 Evaluation of Outcomes

6.2 Comparison with Related Work

6.3 Limitations of the Study

6.4 Practical Implications

6.5 Recommendations and Lessons Learned

7 Conclusion

7.1 Conclusion

7.2 Outlook